KUCCPS IT Ticketing System

Welcome to the official documentation for the **KUCCPS IT Ticketing System**, a comprehensive full-stack support ticketing platform built for the Kenya Universities and Colleges Central Placement Service.

What is this system?

The KUCCPS IT Ticketing System streamlines IT support operations by enabling users to submit support requests, allowing ICT staff to manage and resolve tickets efficiently, and providing powerful analytics, knowledge base management, and integration with external services like Google Workspace.

Key Capabilities

- **Ticket Management** — Full lifecycle from creation to resolution with SLA tracking, escalation, attachments, and audit history
- **User & Department Management** — Role-based access control with `user`, `staff`, and `admin` roles, department assignment, and Active Directory sync
- **Dashboard & Analytics** — Real-time charts, SLA compliance monitoring, agent workload tracking, and CSAT reporting

- **Email Integration** — Automated SMTP notifications, optional IMAP monitoring, and professional email templates
- **Knowledge Base** — Self-service articles with search, ratings, and ticket-suggestion engine
- **Google Workspace Add-on** — Create tickets directly from Gmail using a contextual add-on
- **Status Pages** — Public and internal status issue tracking with updates
- **Custom Forms** — Configurable intake forms for different ticket categories
- **Saved Views** — Reusable ticket filters and column configurations

Who is this for?

Role	Access
Admin	Full system configuration, user management, email templates, LDAP settings
Staff / Agent	Ticket management, department oversight, knowledge base, dashboard analytics
User	Public ticket intake, knowledge base access, status page viewing

Quick Links

- **Source Code:** GitHub Repository
- **License:** ISC
- **Developer:** Felix Ngitari

Documentation Structure

This documentation is organized into the following sections:

1. **Getting Started** — Installation, configuration, and local development setup
2. **Architecture** — System design, data flow, and component interactions
3. **Backend** — Server setup, middleware, controllers, routes, services, and security
4. **Frontend** — Component structure, services, hooks, routing, and styling
5. **Database** — Prisma schema, migrations, seed data, and ORM usage
6. **API Reference** — Complete endpoint documentation for all modules
7. **Google Workspace Add-on** — Add-on setup, deployment, and API integration
8. **Deployment** — Production deployment guides for backend, frontend, and database
9. **Operations** — Maintenance, backup, and monitoring procedures
10. **Troubleshooting** — Common issues and their resolutions

Next
Installation »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Installation

This guide walks you through installing the KUCCPS IT Ticketing System on your local machine for development and testing.

Prerequisites

Before you begin, ensure you have the following installed:

Requirement	Minimum Version	Recommended
Node.js	20.x	22.x
npm	9.x	10.x
PostgreSQL	14.x	16.x
Git	2.x	Latest

Step 1: Clone the Repository

```
git clone https://github.com/Felix273/kuccps-ticketing-system.git
cd kuccps-ticketing
```

Step 2: Install Root Dependencies

```
npm install
```

This installs shared development dependencies like Vite, ESLint, and Tailwind CSS.

Step 3: Install Backend Dependencies

```
cd backend
npm install
cd ..
```

Step 4: Install Frontend Dependencies

```
cd frontend
npm install
cd ..
```

Step 5: Verify Installation

```
# Verify Node.js version
node --version

# Verify npm version
npm --version

# Verify PostgreSQL is running
psql --version
```

Project Structure After Installation

```
kuccps-ticketing/
├── backend/
│   ├── prisma/
│   │   ├── schema.prisma
│   │   ├── seed.js
│   │   └── migrations/
│   ├── scripts/
│   │   └── preflight.js
│   ├── src/
│   │   ├── controllers/
│   │   ├── middleware/
│   │   ├── routes/
│   │   ├── services/
│   │   └── server.js
│   ├── package.json
│   └── .env
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── hooks/
│   │   ├── services/
│   │   ├── utils/
```

```
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── package.json
│   └── .env
├── docs-site/          # Docusaurus documentation
├── google-addon/
│   ├── Code.gs
│   └── appsscript.json
├── package.json
└── vite.config.js
```

Next Steps

After installation, proceed to Configuration to set up your environment variables and database.

Previous
« KUCCPS IT Ticketing System

Next
Configuration »

API Reference

Community

GitHub

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Configuration

The KUCCPS IT Ticketing System uses environment variables for configuration. Never commit real credentials to version control.

Backend Configuration

Create a `.env` file in the `backend/` directory.

Database

```
DATABASE_URL="postgresql://user:password@localhost:5432/database_name?schema=public"
```

Server

```
PORT=5000
NODE_ENV=development
```

JWT Authentication

```
JWT_SECRET=change-this-to-a-long-random-secret
JWT_EXPIRES_IN=7d
```

- `JWT_SECRET` must be at least 32 characters in production.

CORS

```
FRONTEND_URL=http://localhost:5173,http://localhost:5174
```

For local development, both Vite ports are allowed. In production, use HTTPS URLs.

Public Ticket API Key

```
PUBLIC_TICKET_API_KEY=change-this-to-a-long-random-secret
REQUIRE_PUBLIC_TICKET_API_KEY=true
```

- Used by the Google Workspace Add-on to create tickets.
- Must be at least 32 characters in production.

SMTP / Outbound Email

```
EMAIL_SERVICE=gmail
EMAIL_USER=
EMAIL_PASSWORD=
EMAIL_FROM="KUCCPS IT Support <itsupport@example.ac.ke>"
SUPPORT_EMAIL=itsupport@example.ac.ke
```

IMAP / Inbound Email Monitoring

```
IMAP_HOST=imap.gmail.com
IMAP_PORT=993
IMAP_USER=
IMAP_PASSWORD=
IMAP_TLS=true
```

Email monitoring is disabled unless `EMAIL_USER` and `IMAP_USER` are configured.

LDAP / Active Directory

```
USE_LDAP_AUTH=false
LDAP_URL=ldaps://ldap.example.ac.ke:636
LDAP_BIND_DN=
LDAP_BIND_PASSWORD=
LDAP_SEARCH_BASE=
LDAP_SEARCH_FILTER=(uid={{username}})
```

- Use `ldaps://` (LDAPS) in production, never plain `ldap://`.

File Uploads

```
MAX_FILE_SIZE=10485760
MAX_UPLOAD_BYTES=10485760
MAX_KB_UPLOAD_BYTES=5242880
UPLOAD_DIR=./uploads
```

Production Hardening

```
ALLOW_PUBLIC_API_KEY_IN_BODY=false
SMTP_TLS_REJECT_UNAUTHORIZED=true
IMAP_TLS_REJECT_UNAUTHORIZED=true
LDAP_TLS_REJECT_UNAUTHORIZED=true
JSON_BODY_LIMIT=1mb
```

Frontend Configuration

Create a `.env` file in the `frontend/` directory.

```
VITE_API_URL=http://localhost:5000/api
```

For production:

```
VITE_API_URL=https://api.example.ac.ke/api
```

Environment Variable Reference

Variable	Purpose	Required	Default
`DATABASE_URL`	PostgreSQL connection string	Yes	—
`PORT`	Backend server port	No	`5000`
`NODE_ENV`	Runtime environment	No	`development`
`JWT_SECRET`	JWT signing secret	Yes	—
`JWT_EXPIRES_IN`	Token expiration	No	`7d`
`FRONTEND_URL`	Allowed CORS origins	No	`http://localhost:5173,http://localhost:5174`
`PUBLIC_TICKET_API_KEY`	Google Add-on API key	Yes	—

Variable	Purpose	Required	Default
`REQUIRE_PUBLIC_TICKET_API_KEY`	Enforce API key check	No	`true`
`EMAIL_SERVICE`	SMTP provider	No	`gmail`
`EMAIL_USER`	SMTP username	Conditional	—
`EMAIL_PASSWORD`	SMTP password	Conditional	—
`EMAIL_FROM`	Sender display name	No	`KUCCPS IT Support`
`SUPPORT_EMAIL`	Support mailbox	No	`itsupport@kuccps.ac.ke`
`IMAP_HOST`	IMAP server host	Conditional	`imap.gmail.com`
`IMAP_PORT`	IMAP server port	Conditional	`993`
`IMAP_USER`	IMAP username	Conditional	—
`IMAP_PASSWORD`	IMAP password	Conditional	—

Variable	Purpose	Required	Default
`USE_LDAP_AUTH`	Enable LDAP login	No	`false`
`LDAP_URL`	LDAP server URL	Conditional	—
`MAX_FILE_SIZE`	Max upload size in bytes	No	`10485760`
`UPLOAD_DIR`	Upload storage path	No	`./uploads`
`VITE_API_URL`	Frontend API base URL	Yes	—

Configuration Best Practices

1. **Never commit `.env` files** — they are gitignored by default.
2. **Rotate secrets before production** — generate new `JWT_SECRET` and `PUBLIC_TICKET_API_KEY`.
3. **Use HTTPS in production** — update `FRONTEND_URL` and `VITE_API_URL` accordingly.
4. **Use LDAPS, not LDAP** — plain LDAP sends credentials in cleartext.
5. **Configure production CORS explicitly** — avoid wildcard origins.

Previous

« Installation

Next
Development Setup »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Development Setup

This guide covers setting up the development environment, running the application, and preparing the database.

Prerequisites Check

```
# Check Node.js (v20+ required)
node --version

# Check npm (v9+ required)
npm --version

# Check PostgreSQL (v14+ required)
psql --version
```

Step 1: Prepare the Database

Start PostgreSQL and create a database:

```
# Connect to PostgreSQL
psql -U postgres
```

```
# Create database
CREATE DATABASE kuccps_ticketing;
\q
```

Step 2: Configure Environment Variables

Follow the Configuration guide to set up:

- `backend/.env` — database URL, JWT secret, API key
- `frontend/.env` — API base URL

Step 3: Generate Prisma Client

```
cd backend
npx prisma generate
```

This generates the Prisma Client based on `schema.prisma`.

Step 4: Run Database Migrations

```
cd backend
npx prisma migrate dev
```

For production:

```
cd backend
npx prisma migrate deploy
```

Step 5: Run Preflight Checks

```
cd backend
npm run preflight
```

The preflight script validates the environment configuration and database connectivity.

Step 6: Seed the Database

```
cd backend
npx prisma db seed
```

This creates:

Entity	Details
Department	ICT department
Admin user	`admin` / `admin123`
Staff user	`itstaff` / `staff123`

Entity	Details
Default settings	Organization name, colors, SMTP defaults
Email templates	Ticket lifecycle templates

> **Warning:** Change default passwords before using the system beyond local testing.

Step 7: Start the Backend

```
cd backend
npm start
```

The API starts on `http://localhost:5000`.

Verify it is running:

```
curl http://localhost:5000/health
```

Expected response:

```
{
  "status": "OK",
  "message": "KUCCPS Ticketing System API is running"
}
```

Step 8: Start the Frontend

In a new terminal:

```
cd frontend
npm run dev
```

The frontend starts on `http://localhost:5173` or `http://localhost:5174`.

Step 9: Access the Application

Open `http://localhost:5173` in your browser and log in with:

- **Username:** `admin`
- **Password:** `admin123`

Available Scripts

Command	Directory	Purpose
`npm start`	`backend/`	Start Express server
`npm run dev`	`frontend/`	Start Vite dev server
`npm run build`	`frontend/`	Build for production

Command	Directory	Purpose
`npm run preview`	`frontend/`	Preview production build
`npm run lint`	`frontend/`	Run ESLint
`npx prisma studio`	`backend/`	Open Prisma database browser
`npx prisma migrate dev`	`backend/`	Create and apply migrations
`npx prisma db seed`	`backend/`	Seed database with default data

Hot Reloading

- **Backend:** Changes to `server.js` and route files require a server restart.
- **Frontend:** Vite provides instant hot module replacement (HMR) for React components.

Database Browser

Use Prisma Studio to view and edit data:

```
cd backend
npx prisma studio
```

This opens a web UI at `http://localhost:5555`.

Previous
« Configuration

Next
Architecture Overview »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Architecture Overview

This document provides a high-level overview of the KUCCPS IT Ticketing System architecture.

System Diagram

Architecture Layers

1. Client Layer

The system serves two types of clients:

- **Browser / Web App:** A React single-page application that provides the full admin and staff interface.
- **Google Workspace Add-on:** A Gmail contextual add-on that allows users to create tickets directly from Gmail.

2. Frontend Layer

The frontend is a modern React application built with Vite.

Component	Technology	Purpose
UI Framework	React 19	Component-based UI
Build Tool	Vite	Fast development and building
Styling	Tailwind CSS	Utility-first CSS
Icons	Lucide React	Icon library
Charts	Recharts	Analytics visualization
Notifications	React Hot Toast	Toast notifications
Routing	React Router DOM	Client-side routing

3. Backend Layer

The backend is an Express 5 REST API server.

Component	Technology	Purpose
Framework	Express 5	HTTP server and routing
ORM	Prisma	Database access and migrations
Database	PostgreSQL	Data persistence

Component	Technology	Purpose
Auth	JWT + bcrypt	Authentication and password hashing
Email	Nodemailer	Outbound email
Monitoring	IMAP	Inbound email monitoring
Scheduling	node-cron	Background jobs
File Upload	Multer	Attachment handling
LDAP	ldapauth-fork + ldapjs	Active Directory integration

4. Service Layer

Background services that operate independently of HTTP requests:

- **Email Service:** Sends ticket lifecycle notifications via SMTP.
- **IMAP Monitoring:** Polls the support mailbox for incoming emails and processes them as ticket replies.
- **LDAP/AD Sync:** Syncs users and directorates from Active Directory.
- **Scheduler:** Runs cron jobs for SLA checks, daily digests, and other periodic tasks.

5. Data Layer

- **PostgreSQL:** Primary data store for all application data.
- **Prisma ORM:** Type-safe database access with migration management.

- **File Storage:** Local filesystem for ticket attachments and knowledge base materials.

Data Flow: Ticket Creation

Data Flow: Ticket Update and Notification

Key Design Decisions

1. **Separate Frontend and Backend:** Decoupled architecture allows independent scaling and deployment.
2. **Prisma ORM:** Provides type safety, migration management, and database portability.
3. **JWT Authentication:** Stateless authentication suitable for REST APIs.
4. **Public Ticket API:** Allows external intake (Google Add-on) without exposing admin endpoints.
5. **CORS Whitelisting:** Restricts API access to configured frontend origins.
6. **Rate Limiting:** Protects sensitive endpoints from abuse.
7. **Atomic Ticket Numbers:** Uses a mutex to prevent duplicate ticket numbers under concurrent load.

Previous
« Development Setup

Next
Backend Architecture »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Backend Architecture

The backend is a Node.js + Express 5 REST API that handles all business logic, data persistence, and integration with external services.

Directory Structure

```
backend/
├── prisma/
│   ├── schema.prisma          # Database schema definition
│   ├── seed.js                # Seed script for initial data
│   └── migrations/            # Database migration files
├── scripts/
│   ├── preflight.js           # Environment validation script
│   └── update-no-reply-templates.js
├── src/
│   ├── server.js              # Application entry point
│   ├── controllers/           # Request handlers
│   │   ├── authController.js
│   │   ├── ticketController.js
│   │   ├── userController.js
│   │   ├── departmentController.js
│   │   ├── settingsController.js
│   │   ├── knowledgeBaseController.js
│   │   ├── statusIssueController.js
│   │   ├── operationsController.js
```

```
│   │   ├── formController.js
│   │   └── viewController.js
│   ├── routes/                # API route definitions
│   │   ├── auth.js
│   │   ├── tickets.js
│   │   ├── users.js
│   │   ├── departments.js
│   │   ├── settings.js
│   │   ├── knowledgeBase.js
│   │   ├── statusIssues.js
│   │   ├── operations.js
│   │   ├── notifications.js
│   │   ├── forms.js
│   │   ├── views.js
│   │   └── publicTickets.js
│   ├── services/              # Business logic and integrations
│   │   ├── emailService.js
│   │   ├── adService.js
│   │   └── scheduler.js
│   └── middleware/             # Express middleware
│       ├── auth.js
│       └── security.js
├── package.json
└── .env
```

Request Lifecycle

Server Entry Point

The application starts in `backend/src/server.js`.

Key initialization steps:

1. Load environment variables via `dotenv/config`
2. Create Express application
3. Apply security middleware (request ID, headers, CORS, rate limiting)
4. Mount all API route modules
5. Register error handling middleware
6. Start email monitoring if configured
7. Begin listening on the configured port

Route Mounting Order

Routes are mounted in a specific order to ensure correct matching:

1. `/api/auth` — Authentication (with strict rate limiting)
2. `/api/departments` — Department management
3. `/api/users` — User management
4. `/api/notifications` — Notifications
5. `/api/settings` — System settings
6. `/api/views` — Saved views
7. `/api/knowledge-base` — Knowledge base articles
8. `/api/status-issues` — Status issues
9. `/api/forms` — Custom forms
10. `/api/operations` — Dashboard analytics and SLA
11. `/api/public/tickets` — Public ticket intake (with rate limiting)
12. `/api/tickets/public` — Alias for Google Add-on compatibility
13. `/api/tickets` — Staff ticket management (must come after public routes)

Previous
« Architecture Overview

Next
Frontend Architecture »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Frontend Architecture

The frontend is a React 19 single-page application built with Vite and Tailwind CSS. It provides the user interface for all authenticated operations.

Directory Structure

```
frontend/
├── src/
│   ├── main.jsx              # Application entry point
│   ├── App.jsx               # Root component with routing
│   ├── index.css             # Global styles
│   ├── App.css               # App-level styles
│   ├── assets/               # Static assets
│   ├── components/
│   │   ├── auth/             # Login and home pages
│   │   │   ├── HomePage.jsx
│   │   │   └── LoginPage.jsx
│   │   ├── layout/           # Shared layout components
│   │   │   ├── Header.jsx
│   │   │   ├── Navigation.jsx
│   │   │   ├── StatCard.jsx
│   │   │   └── NotificationBell.jsx
│   │   ├── dashboard/        # Dashboard view and charts
│   │   │   ├── DashboardView.jsx
│   │   │   ├── QuickStatsCards.jsx
```

```
│   │   │   ├── TopCategoriesChart.jsx
│   │   │   └── charts/        # Individual chart components
│   │   ├── tickets/           # Ticket management
│   │   │   ├── TicketsView.jsx
│   │   │   ├── TicketCard.jsx
│   │   │   ├── TicketRow.jsx
│   │   │   ├── TicketDetailModal.jsx
│   │   │   ├── TicketAssignModal.jsx
│   │   │   ├── TicketFilters.jsx
│   │   │   ├── QuickFilters.jsx
│   │   │   ├── SavedFilters.jsx
│   │   │   └── AdvancedSearchModal.jsx
│   │   ├── departments/       # Department management
│   │   │   ├── DepartmentsView.jsx
│   │   │   ├── DepartmentCard.jsx
│   │   │   ├── DepartmentModal.jsx
│   │   │   ├── DepartmentDetailModal.jsx
│   │   │   └── UserTicketsDrawer.jsx
│   │   ├── users/             # User management
│   │   │   ├── UsersView.jsx
│   │   │   ├── UserCard.jsx
│   │   │   └── UserModal.jsx
│   │   ├── knowledgebase/     # Knowledge base
│   │   │   └── KnowledgeBaseView.jsx
│   │   ├── admin/             # Admin panel
│   │   │   ├── AdminPanel.jsx
│   │   │   ├── OrganizationSettings.jsx
│   │   │   ├── EmailSettings.jsx
│   │   │   ├── EmailTemplates.jsx
│   │   │   ├── ActiveDirectorySettings.jsx
│   │   │   ├── TicketSettings.jsx
│   │   │   └── CustomFormsSettings.jsx
│   │   ├── settings/          # User settings
│   │   │   └── NotificationPreferences.jsx
│   │   ├── status/            # Status pages
│   │   │   └── StatusPageView.jsx
│   │   └── common/            # Reusable components
│   │       ├── Logo.jsx
```

```
│   │       ├── LoadingSpinner.jsx
│   │       ├── ErrorMessage.jsx
│   │       └── SuccessMessage.jsx
│   ├── services/              # API client modules
│   │   ├── api.js
│   │   ├── authService.js
│   │   ├── ticketService.js
│   │   ├── userService.js
│   │   ├── departmentService.js
│   │   ├── settingsService.js
│   │   ├── knowledgeBaseService.js
│   │   ├── operationsService.js
│   │   ├── statusIssueService.js
│   │   ├── formService.js
│   │   ├── viewService.js
│   │   └── notificationService.js
│   ├── hooks/                 # Custom React hooks
│   │   ├── useAuth.js
│   │   ├── useTickets.js
│   │   ├── useUsers.js
│   │   ├── useDepartments.js
│   │   └── useStatistics.js
│   ├── utils/                 # Utility functions
│   │   ├── constants.js
│   │   ├── exportUtils.js
│   │   └── analytics.js
│   └── styles/
│       └── animations.css
├── public/
│   └── vite.svg
├── package.json
├── vite.config.js
├── tailwind.config.js
├── postcss.config.js
└── index.html
```

Application Flow

Component Communication

Components communicate through a unidirectional data flow:

1. **App.jsx** holds the global state (authentication, active tab, user data).
2. **Services** handle all API communication and return promises.
3. **Hooks** wrap service calls and provide loading/error states.
4. **Components** consume hooks and render UI based on state.

State Management

The application uses React's built-in state management:

- **Local State:** `useState` for component-level state (modals, filters, forms).
- **Derived State:** `useMemo` and `useCallback` for expensive computations.
- **Persistence:** `localStorage` for JWT token and user session.
- **Server State:** Custom hooks (`useTickets`, `useUsers`, etc.) fetch and cache data.

Styling

The application uses **Tailwind CSS** with a custom design system:

- **Brand Colors:** Configurable via system settings (`primaryColor`, `secondaryColor`).

- **Dark Mode:** Toggled via `theme-dark` class on `<body>`.
- **High Contrast:** Toggled via `theme-high-contrast` class.
- **Animations:** Custom CSS animations in `animations.css`.
- **Responsive:** Mobile-first design with Tailwind responsive utilities.

Build Pipeline

Environment Variables

Variable	Purpose	Required
`VITE_API_URL`	Backend API base URL	Yes

Vite exposes only variables prefixed with `VITE_` to the browser.

Previous
« Backend Architecture

Next
Database Architecture »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Database Architecture

The database layer uses PostgreSQL with Prisma ORM for type-safe access and migration management.

Technology Stack

Component	Choice	Purpose
Database	PostgreSQL 14+	Primary data store
ORM	Prisma 5	Type-safe queries and migrations
Migrations	Prisma Migrate	Schema versioning
Seed	Prisma Seed	Initial data population

Entity Relationship Diagram

Key Design Patterns

1. **Cascading Deletes:** Attachments, comments, history, and CSAT responses are deleted when their parent ticket is deleted.
2. **Soft References:** `TicketHistory.changedBy` and `KnowledgeArticle.createdBy` store user identifiers as strings, not foreign keys, preserving history even if users are deleted.
3. **JSON Fields:** `SavedView.filters`, `CustomFormField.options`, and `CustomFormField.conditions` use PostgreSQL JSON for flexible schema.
4. **Indexing:** Key fields like `email`, `username`, `ticketNumber`, `status`, `priority`, and `createdAt` are indexed for query performance.
5. **UUIDs:** All primary keys use UUIDs generated by the database for security and scalability.

Previous
« Frontend Architecture

Next
Google Workspace Add-on »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Google Workspace Add-on

The Google Workspace Add-on allows users to create support tickets directly from Gmail using a contextual add-on interface.

Overview

The add-on is built with **Google Apps Script** and provides:

- **Compose Trigger:** Create tickets from Gmail compose window.
- **Contextual Trigger:** Create tickets from email threads.
- **Form Fields:** Subject, category, priority, and description pre-filled from email context.
- **Ticket Number Display:** Shows the generated ticket number after creation.
- **Email Notification:** Automatically sends a confirmation to the ICT support team.

Directory Structure

```
google-addon/
├── Code.gs           # Main Apps Script code
└── appsscript.json   # Add-on manifest
```

Manifest Configuration

The `appsscript.json` defines the add-on configuration:

```
{
  "timeZone": "Africa/Nairobi",
  "dependencies": {},
  "exceptionLogging": "STACKDRIVER",
  "runtimeVersion": "V8",
  "addOns": {
    "common": {
      "name": "KUCCPS IT Support",
      "logoUrl": "https://example.com/logo.png",
      "useLocaleFromApp": true,
      "supportUrl": "https://example.com/support"
    },
    "gmail": {
      "contextualTriggers": [
        {
          "unconditional": {},
          "onTriggerFunction": "onGmailCompose"
        }
      ],
      "composeTrigger": {
        "draftAccess": "METADATA",
        "onTriggerFunction": "onGmailCompose"
      }
    }
  }
}
```

API Integration

The add-on communicates with the backend through the public ticket API.

Endpoint

```
POST /api/public/tickets
```

Or for Google Add-on compatibility:

```
POST /api/tickets/public
```

Headers

```
X-API-Key: <PUBLIC_TICKET_API_KEY>
Content-Type: application/json
```

Request Body

```
{
  "subject": "Cannot access printer",
  "description": "The printer on the 3rd floor is not responding.",
  "category": "Hardware",
  "priority": "Medium",
  "requesterEmail": "user@example.ac.ke",
  "requesterName": "John Doe"
}
```

Response

```
{
  "success": true,
  "message": "Ticket created successfully",
  "ticket": {
    "id": "uuid",
    "ticketNumber": "TICK20260814-0001",
    "subject": "Cannot access printer",
    "status": "Open",
    "createdAt": "2026-08-14T09:00:00.000Z"
  }
}
```

Apps Script Properties

Store these values in **Apps Script > Project Settings > Script Properties**:

Property	Value	Purpose
`API_URL`	`https://api.example.ac.ke/api`	Backend API base URL
`BACKEND_API_KEY`	Your public ticket API key	Authentication
`SUPPORT_EMAIL`	`itsupport@kuccps.ac.ke`	Support mailbox

Setup Function

Run `setupProductionProperties` once after pasting `Code.gs` if properties are empty:

```
function setupProductionProperties() {
  const properties = PropertiesService.getScriptProperties();
  properties.setProperty('API_URL', 'https://api.example.ac.ke/api');
  properties.setProperty('BACKEND_API_KEY', 'your-api-key-here');
  properties.setProperty('SUPPORT_EMAIL', 'itsupport@kuccps.ac.ke');
}
```

Deployment

1. Open the Apps Script project in Google Workspace.
2. Click **Deploy > Test deployments** to test as yourself.
3. Click **Deploy > New deployment**.
4. Select **Add-on** as the deployment type.
5. Configure deployment settings and install for domain.
6. After every `Code.gs` change, create a new deployment version.

Security

- The add-on uses the `PUBLIC_TICKET_API_KEY` for authentication.
- Set `REQUIRE_PUBLIC_TICKET_API_KEY=true` in production.
- Never expose the API key in client-side code outside the Apps Script environment.
- The backend enforces rate limiting on public ticket endpoints.

Previous
« Database Architecture

Next
Email Service Architecture »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Email Service Architecture

The email service handles all outbound and inbound email communication for the ticketing system.

Components

Outbound Email (SMTP)

Triggering Events

Event	Email Type	Template Key
Ticket created	Confirmation	`ticketCreated`
Ticket status changed	Update	`ticketUpdated`
Ticket resolved	Resolution	`ticketResolved`
Ticket assigned	Assignment	`ticketAssigned`

Event	Email Type	Template Key
New comment added	Notification	`commentAdded`
Overdue SLA alert	Alert	`slaBreached`

Email Templates

Email templates are stored in the `EmailTemplate` model and managed through the admin panel.

Default templates include:

- **Ticket Created:** Confirms receipt and provides ticket number.
- **Ticket Updated:** Notifies requester of status or priority changes.
- **Ticket Resolved:** Announces resolution and requests feedback.

Configuration

SMTP settings are stored in `SystemSettings` and can be updated via `/api/settings`:

```
EMAIL_SERVICE=gmail
EMAIL_USER=itsupport@kuccps.ac.ke
EMAIL_PASSWORD=app-password
EMAIL_FROM="KUCCPS IT Support <itsupport@kuccps.ac.ke>"
SUPPORT_EMAIL=itsupport@kuccps.ac.ke
```

Inbound Email Monitoring (IMAP)

How It Works

1. The backend polls the IMAP mailbox at the configured interval.
2. New emails are parsed to extract subject, body, and attachments.
3. Emails are matched to existing tickets using the subject line or references.
4. Ticket content is updated, or new comments are added.
5. Notifications are sent to assigned staff.

Configuration

```
IMAP_HOST=imap.gmail.com
IMAP_PORT=993
IMAP_USER=itsupport@kuccps.ac.ke
IMAP_PASSWORD=app-password
IMAP_TLS=true
IMAP_POLL_INTERVAL=5
```

Enabling/Disabling

Email monitoring starts automatically on server boot if both `EMAIL_USER` and `IMAP_USER` are configured. It can be toggled via the admin panel.

Email Configuration Model

The `EmailConfig` model tracks configured mailboxes:

Field	Purpose
`provider`	Email provider name (e.g., `gmail`)
`email`	Unique mailbox address
`isActive`	Whether this config is active
`lastChecked`	Last time the mailbox was polled

Security Considerations

1. **TLS Enforcement:** Use `IMAP_TLS_REJECT_UNAUTHORIZED=true` in production.
2. **App Passwords:** For Gmail, use app passwords instead of account passwords.
3. **Firewall Rules:** Ensure the server can reach SMTP (port 587/465) and IMAP (port 993) hosts.
4. **Rate Limits:** The backend applies rate limiting to public endpoints to prevent abuse.

Previous
« Google Workspace Add-on

Next
Backend Overview »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Backend Overview

The backend is a Node.js + Express 5 REST API that powers the KUCCPS IT Ticketing System.

Quick Facts

Aspect	Detail
Runtime	Node.js 20+
Framework	Express 5
ORM	Prisma 5
Database	PostgreSQL 14+
Auth	JWT + bcrypt
Port	5000 (default)

What the Backend Does

- Serves the REST API for all frontend operations
- Manages database access through Prisma ORM
- Handles authentication and authorization
- Sends email notifications via SMTP
- Monitors inbound email via IMAP
- Syncs users from Active Directory
- Runs scheduled background jobs

Key Entry Points

File	Purpose
`src/server.js`	Application entry point, middleware, route mounting
`src/routes/*.js`	API route definitions
`src/controllers/*.js`	Request handlers
`src/services/*.js`	Business logic and integrations
`src/middleware/*.js`	Auth, security, and validation

Base URL

```
http://localhost:5000/api
```

Documentation Sections

- Server Setup — Initialization and middleware
- Middleware — Auth, security, and file upload
- Controllers — Request handlers
- Routes — API endpoint definitions
- Services — Email, AD, and scheduler
- Security — Authentication, authorization, and hardening
- Scheduler — Background jobs

Previous
« Email Service Architecture

Next
Server Setup »

API Reference

Community

GitHub

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Server Setup

The Express server is the core of the backend. This document covers initialization, middleware, and request handling.

Entry Point

File: `backend/src/server.js`

The server initializes in the following order:

1. Load environment variables (`dotenv/config`)
2. Create Express application
3. Validate production configuration
4. Apply security middleware
5. Mount API routes
6. Register error handler
7. Start email monitoring (if configured)
8. Begin listening on the configured port

Middleware Stack

Middleware Details

Middleware	Source	Purpose
`requestId`	`middleware/security.js`	Assigns a unique ID to each request for tracing
`securityHeaders`	`middleware/security.js`	Sets `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy`, `Cross-Origin-Resource-Policy`
`cors`	`cors` package	Restricts origins to configured whitelist
`express.json()`	Express	Parses JSON request bodies (limit: 1mb by default)
`rateLimit`	`express-rate-limit`	Limits requests per IP per time window
`requireStaffOrAdmin`	`middleware/auth.js`	Protects staff/admin routes
`requireAdmin`	`middleware/auth.js`	Protects admin-only routes

Rate Limiting

Rate limits are applied at different levels:

Scope	Window	Max Requests	Purpose
Global	15 minutes	600	General API protection
Auth routes	15 minutes	30	Brute-force protection
Public tickets	15 minutes	30	Spam prevention

Health Check

```
GET /health
```

Response:

```
{
  "status": "OK",
  "message": "KUCCPS Ticketing System API is running"
}
```

The health check dynamically fetches the organization name from `SystemSettings`.

Error Handling

All unhandled errors are caught by the error-handling middleware:

Error Type	Status	Response
CORS origin denied	403	`{ "success": false, "message": "CORS origin denied" }`
File too large	413	`{ "success": false, "message": "Uploaded file is too large" }`
Invalid file type	415	`{ "success": false, "message": "File type is not allowed" }`
Server error	500	`{ "success": false, "message": "Server error" }`

Graceful Shutdown

The server can be stopped gracefully to allow in-flight requests to complete. Currently, the server relies on process signals for shutdown.

Production Validation

`validateProductionConfig()` runs on startup and enforces:

- `NODE_ENV=production` requires explicit CORS origins (no wildcard).
- `REQUIRE_PUBLIC_TICKET_API_KEY=true` requires a valid `PUBLIC_TICKET_API_KEY`.
- `JWT_SECRET` length validation.
- TLS enforcement for LDAP, SMTP, and IMAP when corresponding flags are set.

Previous
« Backend Overview

Next
Middleware »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Middleware

The backend uses custom middleware for authentication, authorization, and security.

Authentication Middleware

File: `backend/src/middleware/auth.js`

`requireStaffOrAdmin`

Protects routes that require `staff` or `admin` roles.

```
const requireStaffOrAdmin = (req, res, next) => {
  const user = req.user;
  if (!user || !['staff', 'admin'].includes(user.role)) {
    return res.status(403).json({ success: false, message: 'Forbidden' });
  }
  next();
};
```

`requireAdmin`

Protects routes that require the `admin` role.

```
const requireAdmin = (req, res, next) => {
  const user = req.user;
  if (!user || user.role !== 'admin') {
    return res.status(403).json({ success: false, message: 'Forbidden' });
  }
  next();
};
```

Token Verification

JWT tokens are verified in the auth middleware:

1. Extract `Authorization` header
2. Verify JWT signature using `JWT_SECRET`
3. Attach decoded user payload to `req.user`
4. Handle expired or invalid tokens with 401 responses

Security Middleware

File: `backend/src/middleware/security.js`

`requestId`

Generates a unique request ID for distributed tracing:

```
const requestId = (req, res, next) => {
  req.id = req.headers['x-request-id'] || crypto.randomUUID();
  res.setHeader('x-request-id', req.id);
  next();
```

```
};
```

`securityHeaders`

Sets security-related HTTP headers:

```
const securityHeaders = (req, res, next) => {
  res.setHeader('X-Content-Type-Options', 'nosniff');
  res.setHeader('X-Frame-Options', 'DENY');
  res.setHeader('Referrer-Policy', 'strict-origin-when-cross-origin');
  res.setHeader('Permissions-Policy', 'camera=(), microphone=(), geolocation=()');
  res.setHeader('Cross-Origin-Resource-Policy', 'same-origin');
  res.setHeader('Cache-Control', 'no-store');
  next();
};
```

`rateLimit`

Configures request rate limiting:

```
const rateLimit = (options) => {
  // options: { windowMs, max, keyPrefix }
};
```

`validateProductionConfig`

Validates environment variables when `NODE_ENV=production`:

```
const validateProductionConfig = () => {
```

```
  // Enforces HTTPS CORS origins
  // Validates JWT_SECRET length
  // Ensures PUBLIC_TICKET_API_KEY is set
  // Validates TLS settings
};
```

File Upload Middleware

Multer handles file uploads for ticket attachments:

```
const upload = multer({
  dest: process.env.UPLOAD_DIR || './uploads',
  limits: { fileSize: process.env.MAX_FILE_SIZE || 10485760 },
  fileFilter: (req, file, cb) => {
    const allowedTypes = (process.env.ALLOWED_FILE_TYPES ||
'jpg,jpeg,png,gif,pdf,doc,docx,xls,xlsx,zip').split(',');
    const ext = path.extname(file.originalname).slice(1).toLowerCase();
    if (allowedTypes.includes(ext)) {
      cb(null, true);
    } else {
      cb(new Error('File type is not allowed'));
    }
  }
});
```

Previous
« Server Setup

Next
Controllers »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Controllers

Controllers handle HTTP requests, validate input, call service layer functions, and return JSON responses.

Response Format

Most API responses follow this standard shape:

```
{
  "success": true,
  "message": "Optional message",
  "data": {}
}
```

Some endpoints return resource-specific keys such as `user`, `users`, `ticket`, `tickets`, `statistics`, `settings`, or `preferences`.

Authentication Controller

File: `backend/src/controllers/authController.js`

Method	Endpoint	Description
`login`	`POST /api/auth/login`	Authenticate user, return JWT
`logout`	`POST /api/auth/logout`	Client-side token cleanup
`getProfile`	`GET /api/auth/profile`	Return current user

Ticket Controller

File: `backend/src/controllers/ticketController.js`

Method	Endpoint	Description
`getTickets`	`GET /api/tickets`	List all tickets with filtering
`getTicket`	`GET /api/tickets/:id`	Get single ticket with relations
`updateTicket`	`PUT /api/tickets/:id`	Update ticket fields
`escalateTicket`	`POST /api/tickets/:id/escalate`	Escalate to another user
`addComment`	`POST /api/tickets/:id/comments`	Add staff comment
`uploadAttachment`	`POST /api/tickets/:id/attachments`	Upload file attachment

User Controller

File: `backend/src/controllers/userController.js`

Method	Endpoint	Description
`getUsers`	`GET /api/users`	List all users
`syncAD`	`POST /api/users/sync/ad`	Sync from Active Directory
`createUser`	`POST /api/users`	Create new user (admin)
`updateUser`	`PUT /api/users/:id`	Update user (admin)
`deleteUser`	`DELETE /api/users/:id`	Delete user (admin)

Department Controller

File: `backend/src/controllers/departmentController.js`

Method	Endpoint	Description
`getDepartments`	`GET /api/departments`	List all departments
`getDepartment`	`GET /api/departments/:id`	Get single department
`syncAD`	`POST /api/departments/sync/ad`	Sync directorates from AD

Method	Endpoint	Description
`createDepartment`	`POST /api/departments`	Create department (admin)
`updateDepartment`	`PUT /api/departments/:id`	Update department (admin)
`deleteDepartment`	`DELETE /api/departments/:id`	Delete department (admin)

Settings Controller

File: `backend/src/controllers/settingsController.js`

Method	Endpoint	Description
`getSettings`	`GET /api/settings`	Get system settings (admin)
`updateSettings`	`PUT /api/settings`	Update system settings (admin)
`getPublicSettings`	`GET /api/settings/public`	Get public branding settings
`getEmailTemplates`	`GET /api/settings/email-templates`	List email templates (admin)

Method	Endpoint	Description
`getEmailTemplate`	`GET /api/settings/email-templates/:type`	Get one template (admin)
`createEmailTemplate`	`POST /api/settings/email-templates`	Create template (admin)
`updateEmailTemplate`	`PUT /api/settings/email-templates/:type`	Update template (admin)
`deleteEmailTemplate`	`DELETE /api/settings/email-templates/:type`	Delete template (admin)
`resetEmailTemplates`	`POST /api/settings/email-templates/reset`	Reset to defaults (admin)

Knowledge Base Controller

File: `backend/src/controllers/knowledgeBaseController.js`

Method	Endpoint	Description
`getPublicArticles`	`GET /api/knowledge-base/public/articles`	List public articles
`getPublicArticle`	`GET /api/knowledge-base/public/articles/:idOrSlug`	Get public article

Method	Endpoint	Description
`getArticles`	`GET /api/knowledge-base/articles`	List all articles (staff/admin)
`getInsights`	`GET /api/knowledge-base/insights`	Get KB insights (staff/admin)
`getSuggestions`	`GET /api/knowledge-base/suggestions/tickets/:ticketId`	Get article suggestions
`createArticle`	`POST /api/knowledge-base/articles`	Create article (staff/admin)
`updateArticle`	`PUT /api/knowledge-base/articles/:id`	Update article (staff/admin)
`deleteArticle`	`DELETE /api/knowledge-base/articles/:id`	Delete article (staff/admin)
`rateArticle`	`POST /api/knowledge-base/articles/:id/rate`	Rate article (public)
`uploadMaterial`	`POST /api/knowledge-base/materials/upload`	Upload material (staff/admin)

Status Issue Controller

File: `backend/src/controllers/statusIssueController.js`

Method	Endpoint	Description
`getPublicIssues`	`GET /api/status-issues/public`	List public status issues
`getIssues`	`GET /api/status-issues`	List all issues (staff/admin)
`createIssue`	`POST /api/status-issues`	Create issue (staff/admin)
`updateIssue`	`PUT /api/status-issues/:id`	Update issue (staff/admin)
`addUpdate`	`POST /api/status-issues/:id/updates`	Add status update (staff/admin)

Operations Controller

File: `backend/src/controllers/operationsController.js`

Method	Endpoint	Description
`getDashboard`	`GET /api/operations/dashboard`	Get dashboard analytics
`getSlaPolicies`	`GET /api/operations/sla-policies`	List SLA policies
`createSlaPolicy`	`POST /api/operations/sla-policies`	Create SLA policy (admin)
`updateSlaPolicy`	`PUT /api/operations/sla-policies/:id`	Update SLA policy (admin)
`submitCsat`	`POST /api/operations/csat`	Submit CSAT response (JWT)

Form Controller

File: `backend/src/controllers/formController.js`

Method	Endpoint	Description
`getPublicForms`	`GET /api/forms/public`	List active public forms
`getForms`	`GET /api/forms`	List forms (staff/admin)
`createForm`	`POST /api/forms`	Create form (admin)
`updateForm`	`PUT /api/forms/:id`	Update form (admin)
`deleteForm`	`DELETE /api/forms/:id`	Delete form (admin)

View Controller

File: `backend/src/controllers/viewController.js`

Method	Endpoint	Description
`getViews`	`GET /api/views`	List saved views
`createView`	`POST /api/views`	Create saved view
`updateView`	`PUT /api/views/:id`	Update saved view

Method	Endpoint	Description
`deleteView`	`DELETE /api/views/:id`	Delete saved view

Previous
« Middleware

Next
Routes »

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Routes

Routes define the URL structure and map HTTP methods to controller functions.

Route Organization

Routes are organized by domain module. Each route file imports its corresponding controller and applies middleware.

Public Ticket Routes

File: `backend/src/routes/publicTickets.js`

```
const express = require('express');
const router = express.Router();
const { createPublicTicket } = require('../controllers/ticketController');

router.post('/', createPublicTicket);

module.exports = router;
```

Authentication Routes

File: `backend/src/routes/auth.js`

```
const express = require('express');
const router = express.Router();
const authController = require('../controllers/authController');

router.post('/login', authController.login);
router.post('/logout', authController.logout);
router.get('/profile', require('../middleware/auth').authenticate,
authController.getProfile);

module.exports = router;
```

Ticket Routes

File: `backend/src/routes/tickets.js`

```
const express = require('express');
const router = express.Router();
const ticketController = require('../controllers/ticketController');
const { requireStaffOrAdmin } = require('../middleware/auth');
const { authenticate } = require('../middleware/auth');

router.get('/', authenticate, requireStaffOrAdmin, ticketController.getTickets);
router.get('/:id', authenticate, requireStaffOrAdmin, ticketController.getTicket);
router.put('/:id', authenticate, requireStaffOrAdmin, ticketController.updateTicket);
router.post('/:id/escalate', authenticate, requireStaffOrAdmin,
ticketController.escalateTicket);
router.post('/:id/comments', authenticate, requireStaffOrAdmin,
ticketController.addComment);
router.post('/:id/attachments', authenticate, requireStaffOrAdmin,
  upload.single('file'), ticketController.uploadAttachment);
```

```
module.exports = router;
```

Mounting Order in server.js

Routes are mounted in `backend/src/server.js`:

```
app.use('/api/auth', rateLimit({ windowMs: 15 * 60 * 1000, max: 30, keyPrefix: 'auth'
}), authRoutes);
app.use('/api/departments', departmentRoutes);
app.use('/api/users', userRoutes);
app.use('/api/notifications', require('./routes/notifications'));
app.use('/api/settings', require('./routes/settings'));
app.use('/api/views', require('./routes/views'));
app.use('/api/knowledge-base', require('./routes/knowledgeBase'));
app.use('/api/status-issues', require('./routes/statusIssues'));
app.use('/api/forms', require('./routes/forms'));
app.use('/api/operations', require('./routes/operations'));
app.use('/api/public/tickets', rateLimit(...), publicTicketRoutes);
app.use('/api/tickets/public', rateLimit(...), publicTicketRoutes);
app.use('/api/tickets', ticketRoutes);
```

> **Important:** Public ticket routes must be mounted before the general `/api/tickets` route to avoid path conflicts.

Route Patterns

Pattern	Example	Purpose
`GET /`	`GET /api/tickets`	List resources

Pattern	Example	Purpose
`GET /:id`	`GET /api/tickets/123`	Get single resource
`POST /`	`POST /api/tickets`	Create resource
`PUT /:id`	`PUT /api/tickets/123`	Update resource
`DELETE /:id`	`DELETE /api/tickets/123`	Delete resource
`POST /:id/action`	`POST /api/tickets/123/escalate`	Resource-specific action

Previous
« Controllers

Next
Services »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Services

Services contain the business logic and integration code that controllers call.

Email Service

File: `backend/src/services/emailService.js`

Responsibilities

- Sending ticket lifecycle emails via SMTP
- Starting and managing IMAP monitoring
- Processing incoming emails and creating ticket replies

Key Functions

Function	Purpose
`sendTicketCreatedEmail(ticket)`	Send confirmation when ticket is created
`sendTicketUpdatedEmail(ticket, changes)`	Notify requester of status/priority changes

Function	Purpose
`sendTicketResolvedEmail(ticket)`	Announce resolution and request CSAT
`startEmailMonitoring()`	Begin IMAP polling loop
`processIncomingEmail(email)`	Parse and route incoming email to ticket

Email Template Resolution

Templates are loaded from the `EmailTemplate` model in the database. If a template is missing, the service falls back to a hardcoded default.

Active Directory Service

File: `backend/src/services/adService.js`

Responsibilities

- Authenticating users against LDAP/Active Directory
- Syncing users from AD to the local database
- Syncing directorates (organizational units/groups) as departments

Key Functions

Function	Purpose
`authenticateLDAP(username, password)`	Validate credentials against AD
`syncUsers()`	Import users from AD
`syncDirectorates()`	Import directorates as departments

LDAP Configuration

LDAP settings are stored in `SystemSettings` and include:

- `ldapUrl` — LDAPS server URL
- `ldapBindDn` — Service account DN
- `ldapBindPassword` — Service account password
- `ldapUserSearchBase` — Base DN for user searches
- `ldapUserFilter` — LDAP filter for user objects
- `ldapUsernameAttribute` — Attribute mapped to username
- `ldapEmailAttribute` — Attribute mapped to email
- `ldapNameAttribute` — Attribute mapped to display name

Scheduler

File: `backend/src/services/scheduler.js`

Responsibilities

- Running periodic background jobs using `node-cron`

Scheduled Jobs

Job	Schedule	Purpose
SLA Check	Every 5 minutes	Identify overdue tickets and create notifications
Daily Digest	Daily at 8:00 AM	Send daily ticket summary to staff
AD Sync	Daily at 2:00 AM	Sync users and directorates from AD (if enabled)

Job Lifecycle

Previous
« Routes

Next
Security »

API Reference

Community

GitHub

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Security

The backend implements multiple layers of security to protect data and prevent abuse.

Authentication

JWT Bearer Tokens

- Users authenticate via `POST /api/auth/login`.
- The server issues a JWT signed with `JWT_SECRET`.
- Tokens are sent in the `Authorization` header: `Bearer <token>`.
- Tokens expire based on `JWT_EXPIRES_IN` (default: 7 days).

Password Hashing

- Passwords are hashed using `bcrypt` before storage.
- The `User` model stores only the hash, never the plaintext password.
- Login compares the provided password against the stored hash.

Session Management

- JWTs are stateless; no server-side session storage is required.

- The frontend stores the token in `localStorage`.
- On auth failure, the client removes the token and redirects to login.

Authorization

Role-Based Access Control

Role	Access
`user`	Public intake, knowledge base, status pages
`staff`	Ticket management, dashboard, knowledge base, users, departments
`admin`	Full access including settings, user CRUD, email templates, LDAP config

Route Protection

- `requireStaffOrAdmin` — Staff and admin access only.
- `requireAdmin` — Admin access only.
- Public routes are explicitly whitelisted.

Transport Security

CORS

- Origins are restricted to `CORS_ORIGINS` or `FRONTEND_URL`.
- Credentials are allowed for authenticated requests.
- In production, wildcard origins are rejected by `validateProductionConfig`.

Security Headers

Header	Value	Purpose
`X-Content-Type-Options`	`nosniff`	Prevent MIME type sniffing
`X-Frame-Options`	`DENY`	Prevent clickjacking
`Referrer-Policy`	`strict-origin-when-cross-origin`	Control referrer leakage
`Permissions-Policy`	`camera=(), microphone=(), geolocation=()`	Disable unnecessary browser features
`Cross-Origin-Resource-Policy`	`same-origin`	Prevent cross-origin reading
`Cache-Control`	`no-store`	Prevent caching of sensitive responses

Request Size Limits

- JSON body: 1mb (configurable via `JSON_BODY_LIMIT`)
- File uploads: 10mb (configurable via `MAX_FILE_SIZE`)

Input Validation

- Express JSON parser rejects malformed bodies.
- `express-validator` is available for route-level validation.
- Prisma parameterized queries prevent SQL injection.
- File type checking prevents malicious uploads.

Rate Limiting

Scope	Window	Max Requests
Global	15 min	600
Auth endpoints	15 min	30
Public ticket intake	15 min	30

Public API Protection

- `PUBLIC_TICKET_API_KEY` authenticates public ticket creation.
- `REQUIRE_PUBLIC_TICKET_API_KEY=true` enforces the check in production.
- The key is validated against the server configuration, not a database record.

Production Hardening

When `NODE_ENV=production`:

- CORS must use explicit HTTPS origins.
- `JWT_SECRET` must be at least 32 characters.
- `PUBLIC_TICKET_API_KEY` must be at least 32 characters.
- TLS is enforced for LDAP (`ldaps://`), SMTP, and IMAP.
- `x-powered-by` header is disabled.

Previous
« Services

Next
Scheduler »

GitHub

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Scheduler

The scheduler runs periodic background jobs using `node-cron`.

Architecture

Jobs

SLA Check

Schedule: Every 5 minutes

Function: Queries tickets where `slaDueAt < now()` and `slaBreached = false`. Marks them as breached and creates notifications for assigned staff.

Impact: Ensures SLA compliance is monitored in real-time.

Daily Digest

Schedule: 8:00 AM EAT daily

Function: Generates a summary of ticket activity for the previous day and emails it to staff members.

Impact: Keeps staff informed without requiring them to check the dashboard.

Active Directory Sync

Schedule: 2:00 AM EAT daily

Function: Syncs users and directorates from AD when `ldapEnabled` is true.

Impact: Keeps local user directory in sync with organizational changes.

Job Management

Jobs are initialized when the server starts. If a job fails, the error is logged but does not prevent other jobs from running.

Manual Triggers

Some operations can be triggered manually via API endpoints:

- **Test Email:** `POST /api/notifications/test` — Sends a test notification email.
- **AD Sync:** `POST /api/users/sync/ad` and `POST /api/departments/sync/ad` — Manual sync triggers.

Previous
« Security

Next
Frontend Overview »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Frontend Overview

The frontend is a React 19 single-page application that serves as the primary interface for ICT staff and administrators.

Technology Stack

Technology	Version	Purpose
React	19.2	UI framework
Vite	7.2	Build tool and dev server
Tailwind CSS	3.4	Utility-first CSS
Lucide React	0.554	Icon library
Recharts	3.4	Data visualization
React Router DOM	Latest	Client-side routing
React Hot Toast	Latest	Toast notifications

Entry Points

main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

App.jsx

The root component manages:

- Authentication state
- Active tab/navigation
- Global data fetching (tickets, statistics)
- Branding application from public settings

Tab-Based Navigation

The authenticated interface uses a tab-based layout:

Tab	Component	Role Access
Dashboard	`DashboardView`	Staff, Admin
Tickets	`TicketsView`	Staff, Admin
Departments	`DepartmentsView`	Staff, Admin
Users	`UsersView`	Staff, Admin
Knowledge Base	`KnowledgeBaseView`	Staff, Admin
Admin	`AdminPanel`	Admin only

Data Fetching Strategy

The application uses a custom hook-based data fetching pattern:

Component Hierarchy

State Management

The application uses React's built-in state management without external libraries:

- **Local State:** `useState` for UI state (modals, filters, forms)

- **Derived State:** `useMemo` for expensive calculations (filtered ticket lists)
- **Side Effects:** `useEffect` for data fetching and branding application
- **Persistence:** `localStorage` for JWT token and user session

Previous
« Scheduler

Next
Frontend Structure »

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Frontend Structure

The frontend source code is organized into a clear hierarchy of directories.

Directory Layout

```
frontend/
├── src/
│   ├── main.jsx                    # React entry point
│   ├── App.jsx                     # Root component + routing
│   ├── index.css                   # Tailwind directives + global styles
│   ├── App.css                     # App-specific styles
│   ├── assets/                     # Static assets
│   │   └── react.svg
│   ├── components/                 # React components
│   │   ├── auth/                   # Authentication pages
│   │   ├── layout/                 # Shared layout
│   │   ├── dashboard/              # Dashboard and charts
│   │   ├── tickets/                # Ticket management
│   │   ├── departments/            # Department management
│   │   ├── users/                  # User management
│   │   ├── knowledgebase/          # Knowledge base
│   │   ├── admin/                  # Admin settings
│   │   ├── settings/               # User preferences
│   │   ├── status/                 # Status pages
│   │   └── common/                 # Shared UI components
```

```
│   ├── services/                 # API client modules
│   ├── hooks/                    # Custom React hooks
│   ├── utils/                    # Utilities and constants
│   └── styles/                   # Additional CSS
├── public/
│   └── vite.svg
├── package.json
├── vite.config.js
├── tailwind.config.js
├── postcss.config.js
└── index.html
```

Module Responsibilities

components/

Contains all presentational and container components. Organized by feature domain:

- **auth:** Login and landing pages
- **layout:** Header, navigation, stat cards, notification bell
- **dashboard:** Dashboard view, quick stats, chart components
- **tickets:** Ticket list, detail modal, assignment, filters, search
- **departments:** Department cards, modals, user tickets drawer
- **users:** User cards, user modal
- **knowledgebase:** Knowledge base view
- **admin:** Admin panel with all settings sections
- **settings:** Notification preferences
- **status:** Public status page
- **common:** Logo, loading spinner, error/success messages

services/

Each service module encapsulates API communication for a specific domain:

- **api.js:** Central fetch client with JWT header injection and auth redirect
- **authService.js:** Login, logout, profile operations
- **ticketService.js:** Ticket CRUD, comments, attachments, escalation
- **userService.js:** User management
- **departmentService.js:** Department CRUD
- **settingsService.js:** System settings and email templates
- **knowledgeBaseService.js:** KB articles and ratings
- **operationsService.js:** Dashboard analytics and SLA policies
- **statusIssueService.js:** Status issues
- **formService.js:** Custom forms
- **viewService.js:** Saved views
- **notificationService.js:** Notifications and preferences

hooks/

Custom hooks wrap service calls and provide loading/error states:

- **useAuth.js:** Authentication state helpers
- **useTickets.js:** Ticket fetching with pagination
- **useUsers.js:** User fetching
- **useDepartments.js:** Department fetching
- **useStatistics.js:** Dashboard statistics fetching

utils/

Utility functions and constants:

- **constants.js:** Application constants (categories, priorities, statuses)
- **exportUtils.js:** CSV/Excel export functionality
- **analytics.js:** Analytics helper functions

Previous
« Frontend Overview

Next
Components »

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Components

This document describes the key frontend components and their responsibilities.

Layout Components

Header

File: `frontend/src/components/layout/Header.jsx`

Displays the organization logo, current user information, and logout button. Also handles active tab changes.

Navigation

File: `frontend/src/components/layout/Navigation.jsx`

Renders the tab-based navigation bar with icons and labels. Highlights the active tab.

StatCard

File: `frontend/src/components/layout/StatCard.jsx`

Reusable card component for displaying statistics with icons, labels, and trend indicators.

NotificationBell

File: `frontend/src/components/layout/NotificationBell.jsx`

Displays unread notification count and provides access to the notification dropdown.

Dashboard Components

DashboardView

File: `frontend/src/components/dashboard/DashboardView.jsx`

The main dashboard container. Fetches and displays:

- Quick statistics cards
- Interactive charts
- SLA compliance badges

QuickStatsCards

File: `frontend/src/components/dashboard/QuickStatsCards.jsx`

Grid of stat cards showing key metrics (total tickets, open, resolved, overdue).

Charts

Located in `frontend/src/components/dashboard/charts/`:

Component	Chart Type	Data
`TicketTrendsChart.jsx`	Area/Line	Ticket volume over time
`VolumeByHourChart.jsx`	Bar	Tickets by hour of day
`TopCategoriesChart.jsx`	Bar	Top issue categories
`PriorityChart.jsx`	Bar	Priority distribution
`StatusPieChart.jsx`	Pie	Status breakdown
`ActiveVsResolvedChart.jsx`	Area	Active vs resolved trends
`DepartmentStatsChart.jsx`	Bar	Department ticket counts
`DepartmentPerformanceChart.jsx`	Bar	Department resolution rates
`DepartmentTicketChart.jsx`	Pie	Department ticket distribution
`UserWorkloadChart.jsx`	Bar	Agent workload
`TopPerformersChart.jsx`	Bar	Top performing agents
`EmailDomainChart.jsx`	Bar	Requester email domains
`TopRequestersChart.jsx`	Bar	Top ticket requesters
`AssignmentChart.jsx`	Pie	Ticket assignment stats

Component	Chart Type	Data
`ResponseTimeChart.jsx`	Bar	Average response times
`ResolutionRateChart.jsx`	Line	Resolution rate trends

Ticket Components

TicketsView

File: `frontend/src/components/tickets/TicketsView.jsx`

Main ticket list container with filters, search, and refresh.

TicketCard / TicketRow

Display individual tickets in card or table row format.

TicketDetailModal

Opens a modal showing full ticket details, comments, attachments, and history.

TicketAssignModal

Modal for assigning or reassigning tickets to staff members.

TicketFilters

Advanced filtering controls for the ticket list.

QuickFilters

One-click filter buttons for common views (My Tickets, Overdue, Unassigned).

SavedFilters

Manage and apply saved ticket filter configurations.

AdvancedSearchModal

Full-text search across ticket fields.

Department Components

DepartmentsView

Main department list and management interface.

DepartmentCard

Displays department information with ticket counts and actions.

DepartmentModal

Create/edit department form.

DepartmentDetailModal

Detailed department view with member list and ticket stats.

UserTicketsDrawer

Slide-out drawer showing all tickets for a specific user within a department.

User Components

UsersView

User list and management interface.

UserCard

Displays user information with role and actions.

UserModal

Create/edit user form.

Knowledge Base Components

KnowledgeBaseView

Self-service knowledge base with search, categories, and article ratings.

Admin Components

AdminPanel

Container for all admin settings sections with tab navigation.

Component	File	Purpose
`OrganizationSettings`	`admin/OrganizationSettings.jsx`	Org name, colors, logo
`EmailSettings`	`admin/EmailSettings.jsx`	SMTP/IMAP configuration
`EmailTemplates`	`admin/EmailTemplates.jsx`	Manage email templates
`ActiveDirectorySettings`	`admin/ActiveDirectorySettings.jsx`	LDAP configuration
`TicketSettings`	`admin/TicketSettings.jsx`	Default ticket values
`CustomFormsSettings`	`admin/CustomFormsSettings.jsx`	Manage custom forms

Common Components

Logo

Renders the organization logo with fallback to text.

LoadingSpinner

Animated loading indicator.

ErrorMessage / SuccessMessage

Display operational feedback.

Previous
« Frontend Structure

Next
Services »

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Services

The frontend service layer provides a clean abstraction over the backend API.

Central API Client

File: `frontend/src/services/api.js`

Responsibilities

- Centralizes all HTTP requests
- Injects JWT token into request headers
- Handles authentication errors (401/403)
- Parses JSON responses
- Redirects to login on auth failure

Usage

```
import { api } from '../services/api';

const response = await api.get('/tickets');
const data = await response.json();
```

Configuration

The API base URL is read from `VITE_API_URL`:

```
const API_BASE = import.meta.env.VITE_API_URL || 'http://localhost:5000/api';
```

Service Modules

authService.js

Handles authentication operations:

Function	Method	Endpoint
`login(username, password)`	POST	`/auth/login`
`logout()`	POST	`/auth/logout`
`getProfile()`	GET	`/auth/profile`
`getCurrentUser()`	—	Reads from localStorage

ticketService.js

Ticket operations:

Function	Method	Endpoint
`fetchTickets()`	GET	`/tickets`
`fetchTicket(id)`	GET	`/tickets/:id`
`updateTicket(id, data)`	PUT	`/tickets/:id`
`escalateTicket(id, userId)`	POST	`/tickets/:id/escalate`
`addComment(id, content, isInternal)`	POST	`/tickets/:id/comments`
`uploadAttachment(id, file)`	POST	`/tickets/:id/attachments`

userService.js

User management:

Function	Method	Endpoint
`fetchUsers()`	GET	`/users`
`createUser(data)`	POST	`/users`
`updateUser(id, data)`	PUT	`/users/:id`
`deleteUser(id)`	DELETE	`/users/:id`

Function	Method	Endpoint
`syncAD()`	POST	`/users/sync/ad`

departmentService.js

Department management:

Function	Method	Endpoint
`fetchDepartments()`	GET	`/departments`
`fetchDepartment(id)`	GET	`/departments/:id`
`createDepartment(data)`	POST	`/departments`
`updateDepartment(id, data)`	PUT	`/departments/:id`
`deleteDepartment(id)`	DELETE	`/departments/:id`
`syncAD()`	POST	`/departments/sync/ad`

settingsService.js

System settings:

Function	Method	Endpoint
`getSettings()`	GET	`/settings`
`updateSettings(data)`	PUT	`/settings`
`getPublicSettings()`	GET	`/settings/public`
`fetchEmailTemplates()`	GET	`/settings/email-templates`
`updateEmailTemplate(type, data)`	PUT	`/settings/email-templates/:type`

knowledgeBaseService.js

Knowledge base:

Function	Method	Endpoint
`fetchPublicArticles()`	GET	`/knowledge-base/public/articles`
`fetchArticles()`	GET	`/knowledge-base/articles`
`fetchArticle(idOrSlug)`	GET	`/knowledge-base/articles/:idOrSlug`
`createArticle(data)`	POST	`/knowledge-base/articles`
`updateArticle(id, data)`	PUT	`/knowledge-base/articles/:id`

Function	Method	Endpoint
`deleteArticle(id)`	DELETE	`/knowledge-base/articles/:id`
`rateArticle(id, helpful)`	POST	`/knowledge-base/articles/:id/rate`

operationsService.js

Analytics and operations:

Function	Method	Endpoint
`fetchDashboard()`	GET	`/operations/dashboard`
`fetchSlaPolicies()`	GET	`/operations/sla-policies`
`createSlaPolicy(data)`	POST	`/operations/sla-policies`
`updateSlaPolicy(id, data)`	PUT	`/operations/sla-policies/:id`
`submitCsat(data)`	POST	`/operations/csat`

statusIssueService.js

Status issues:

Function	Method	Endpoint
`fetchPublicIssues()`	GET	`/status-issues/public`
`fetchIssues()`	GET	`/status-issues`
`createIssue(data)`	POST	`/status-issues`
`updateIssue(id, data)`	PUT	`/status-issues/:id`

formService.js

Custom forms:

Function	Method	Endpoint
`fetchPublicForms()`	GET	`/forms/public`
`fetchForms()`	GET	`/forms`
`createForm(data)`	POST	`/forms`
`updateForm(id, data)`	PUT	`/forms/:id`
`deleteForm(id)`	DELETE	`/forms/:id`

viewService.js

Saved views:

Function	Method	Endpoint
`fetchViews()`	GET	`/views`
`createView(data)`	POST	`/views`
`updateView(id, data)`	PUT	`/views/:id`
`deleteView(id)`	DELETE	`/views/:id`

notificationService.js

Notifications:

Function	Method	Endpoint
`fetchNotifications()`	GET	`/notifications`
`markAllRead()`	PUT	`/notifications/read/all`
`markRead(id)`	PUT	`/notifications/:id/read`
`dismiss(id)`	DELETE	`/notifications/:id`
`fetchPreferences()`	GET	`/notifications/preferences`
`updatePreferences(data)`	PUT	`/notifications/preferences`

Function	Method	Endpoint
`sendTestEmail()`	POST	`/notifications/test`

Previous
« Components

Next
Hooks »

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Hooks

Custom React hooks encapsulate data fetching and state management logic.

useAuth

File: `frontend/src/hooks/useAuth.js`

Manages authentication state:

- Reads JWT token from `localStorage`
- Provides `isAuthenticated` boolean
- Provides `currentUser` object
- Handles logout cleanup

```
const { isAuthenticated, currentUser, logout } = useAuth();
```

useTickets

File: `frontend/src/hooks/useTickets.js`

Manages ticket data fetching:

```
const { tickets, isLoading, error, fetchTickets } = useTickets(isAuthenticated);
```

- Fetches tickets when `isAuthenticated` becomes true
- Provides loading and error states
- Exposes `fetchTickets` for manual refresh

useUsers

File: `frontend/src/hooks/useUsers.js`

Manages user data fetching:

```
const { users, isLoading, error, fetchUsers } = useUsers(isAuthenticated);
```

useDepartments

File: `frontend/src/hooks/useDepartments.js`

Manages department data fetching:

```
const { departments, isLoading, error, fetchDepartments } =
useDepartments(isAuthenticated);
```

useStatistics

File: `frontend/src/hooks/useStatistics.js`

Manages dashboard statistics fetching:

```
const { statistics, isLoading, error, fetchStatistics } =
useStatistics(isAuthenticated);
```

Hook Lifecycle

Dependency Management

Hooks use `useEffect` with `isAuthenticated` as a dependency to trigger data fetching when authentication state changes.

Previous
« Services

Next
Routing »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Routing

The frontend uses React Router DOM for client-side navigation.

Router Configuration

File: `frontend/src/App.jsx`

The application does not use a `<BrowserRouter>` with explicit routes. Instead, it uses a **tab-based architecture** where the active view is determined by the `activeTab` state variable.

```
const [activeTab, setActiveTab] = useState('dashboard');

// Render based on active tab
{activeTab === 'dashboard' && <DashboardView ... />}
{activeTab === 'tickets' && <TicketsView ... />}
{activeTab === 'departments' && <DepartmentsView />}
{activeTab === 'users' && <UsersView />}
{activeTab === 'knowledgebase' && <KnowledgeBaseView />}
{activeTab === 'admin' && <AdminPanel />}
```

Navigation Flow

Modal-Based Navigation

Many features use modals instead of route changes:

- **TicketDetailModal:** Opens from TicketsView to show ticket details
- **TicketAssignModal:** Opens from TicketDetailModal for assignment
- **DepartmentModal:** Opens from DepartmentsView for create/edit
- **UserModal:** Opens from UsersView for create/edit
- **AdvancedSearchModal:** Opens from TicketsView for search

Public Routes

Path	Component	Access
`/`	`HomePage`	Public
`/login`	`LoginPage`	Public
`/knowledge-base`	`KnowledgeBaseView`	Public (limited)

Protected Routes

All other views require authentication. The `App.jsx` component checks for a valid token before rendering the authenticated layout.

Deep Linking

Currently, the application does not support deep linking to specific tabs via URL. The active tab is managed entirely through component state. Future enhancement could involve URL-based routing for shareable links.

Previous
« Hooks

Next
Styling »

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Styling

The frontend uses Tailwind CSS with a custom design system for consistent branding and responsive design.

Tailwind Configuration

File: `frontend/tailwind.config.js`

The Tailwind configuration extends the default theme with:

- Custom brand colors
- Animation keyframes
- Extended font families
- Dark mode support

Design System

Brand Colors

Brand colors are configurable through `SystemSettings` and applied dynamically:

```
document.documentElement.style.setProperty('--brand-primary', settings.primaryColor ||
'#911414');
document.documentElement.style.setProperty('--brand-secondary', settings.secondaryColor
|| '#d20001');
```

Default Palette

Color	Hex	Usage
Primary	`#911414`	KUCCPS brand red, buttons, headers
Secondary	`#d20001`	Accent color, hover states
Dark	`#1f2937`	Dark mode backgrounds
Gray	`#6b7280`	Secondary text, borders

CSS Architecture

Global Styles (`index.css`)

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  :root {
    --brand-primary: #911414;
```

```
    --brand-secondary: #d20001;
  }
}

@layer components {
  .btn-primary {
    @apply px-4 py-2 rounded-lg text-white font-medium;
    background-color: var(--brand-primary);
  }

  .card {
    @apply bg-white rounded-xl shadow-sm border border-gray-200 p-6;
  }
}
```

Animations (`styles/animations.css`)

Custom animations for:

- Fade in/out
- Slide in from bottom
- Spinner rotation
- Pulse effects for notifications

Responsive Design

The application uses a mobile-first approach with Tailwind responsive prefixes:

Breakpoint	Width	Target
`sm`	640px	Large phones
`md`	768px	Tablets
`lg`	1024px	Small laptops
`xl`	1280px	Desktops
`2xl`	1536px	Large screens

Theme Variants

Dark Mode

Enabled via `darkModeEnabled` in `SystemSettings`:

```
.theme-dark {
  background-color: #1f2937;
  color: #f9fafb;
}
```

High Contrast Mode

Enabled via `contrastMode` in `SystemSettings`:

```
.theme-high-contrast {
  filter: contrast(1.5);
}
```

PostCSS Pipeline

File: `postcss.config.js`

```
export default {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
};
```

Autoprefixer automatically adds vendor prefixes for cross-browser compatibility.

Previous
« Routing

Next
Database Overview »

Getting Started

Architecture

API Reference

Community

GitHub

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Database Overview

The database layer uses PostgreSQL with Prisma ORM for type-safe access, migrations, and seeding.

Technology Stack

Component	Technology	Purpose
Database	PostgreSQL	Primary data store
ORM	Prisma 5	Type-safe database access
Migrations	Prisma Migrate	Schema version control
Seeding	Prisma Seed	Initial data population
Browser	Prisma Studio	Visual database browser

Connection

The database connection is configured via `DATABASE_URL` in `backend/.env`:

```
DATABASE_URL="postgresql://user:password@localhost:5432/kuccps_ticketing?schema=public"
```

Prisma connects to the database using this URL and manages connection pooling automatically.

Prisma Client

The Prisma Client is generated from the schema:

```
cd backend
npx prisma generate
```

The generated client provides a type-safe API:

```
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

const tickets = await prisma.ticket.findMany({
  where: { status: 'Open' },
  include: { assignedTo: true, department: true }
});
```

Migrations

Migrations are managed through Prisma Migrate:

```
# Create a new migration
cd backend
npx prisma migrate dev --name description_of_change

# Apply migrations in development
cd backend
npx prisma migrate dev

# Apply migrations in production
cd backend
npx prisma migrate deploy
```

Seeding

Seed data is defined in `backend/prisma/seed.js`:

```
cd backend
npx prisma db seed
```

The seed script creates default data for local development.

Prisma Studio

Prisma Studio provides a visual interface for browsing and editing data:

```
cd backend
npx prisma studio
```

This opens a web UI at `http://localhost:5555`.

Previous
« Styling

Next
Database Schema »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Database Schema

The database schema is defined in `backend/prisma/schema.prisma` and consists of 15 models.

Model Summary

Model	Purpose	Key Fields
`User`	Application users	email, username, role, departmentId
`Department`	Organizational units	name, code
`Ticket`	Support tickets	ticketNumber, status, priority, category
`Attachment`	Ticket files	filename, filepath, mimetype, size
`Comment`	Ticket comments	content, isInternal
`TicketHistory`	Audit trail	field, oldValue, newValue
`SystemSettings`	Global configuration	organizationName, smtp settings, ldap settings

Model	Purpose	Key Fields
`EmailTemplate`	Email templates	type, subject, html
`SavedView`	Saved ticket filters	name, filters, visibility
`KnowledgeArticle`	KB articles	title, slug, content, status
`StatusIssue`	Status incidents	title, status, severity
`StatusIssueUpdate`	Status updates	message, status
`CustomForm`	Intake forms	name, isActive
`CustomFormField`	Form fields	label, key, type, required
`SlaPolicy`	SLA policies	responseMinutes, resolutionMinutes
`CsatResponse`	CSAT ratings	rating, comment
`NotificationPreference`	User notification settings	emailOnAssignment, dailyDigest
`EmailConfig`	Mailbox configuration	provider, email, isActive

Key Relationships

Indexing Strategy

The schema defines indexes on frequently queried fields:

```
@@index([email])
@@index([username])
@@index([ticketNumber])
@@index([status])
@@index([priority])
@@index([requesterEmail])
@@index([createdAt])
```

These indexes improve query performance for common filters like ticket search and user lookup.

Enums and Constraints

- **Role:** `user`, `staff`, `admin` (stored as `String` with validation)
- **Status:** `Open`, `In Progress`, `Resolved`, `Closed`
- **Priority:** `Low`, `Medium`, `High`, `Critical`
- **Severity:** `Informational`, `Warning`, `Error`
- **Visibility:** `private`, `shared`, `global`

Cascading Deletes

```
Attachment {
  ticket Ticket @relation(fields: [ticketId], references: [id], onDelete: Cascade)
```

```
}
```

When a ticket is deleted, its attachments, comments, history, and CSAT responses are automatically deleted.

Previous
« Database Overview

Next
Migrations »

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Migrations

Prisma Migrate manages database schema changes through versioned migration files.

Migration Files

Migrations are stored in `backend/prisma/migrations/` with timestamp-based naming:

```
backend/prisma/migrations/
├── 20250101000000_init/
│   └── migration.sql
├── 20250102000000_add_email_templates/
│   └── migration.sql
└── ...
```

Creating Migrations

```
cd backend
npx prisma migrate dev --name descriptive_change_name
```

This command:

1. Saves the migration SQL to `migrations/`
2. Applies the migration to your local database
3. Regenerates the Prisma Client

Migration SQL

Migration files contain raw SQL that transforms the schema:

```sql
-- CreateTable
CREATE TABLE "Ticket" (
    "id" TEXT NOT NULL,
    "ticketNumber" TEXT NOT NULL,
    "subject" TEXT NOT NULL,
    "status" TEXT NOT NULL DEFAULT 'Open',
    "priority" TEXT NOT NULL DEFAULT 'Medium',
    "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT "Ticket_pkey" PRIMARY KEY ("id")
);

-- CreateIndex
CREATE UNIQUE INDEX "Ticket_ticketNumber_key" ON "Ticket"("ticketNumber");

-- CreateIndex
CREATE INDEX "Ticket_status_idx" ON "Ticket"("status");
```

Applying Migrations

Development

```
cd backend
npx prisma migrate dev
```

Production

```
cd backend
npx prisma migrate deploy
```

> **Important:** Never use `prisma migrate dev` in production. It creates new migration files, which should only happen during development.

Resetting the Database

```
cd backend
npx prisma migrate reset
```

This drops the database, recreates it, applies all migrations, and runs the seed script. Use with caution in production.

Migration Best Practices

1. **Meaningful names:** Use descriptive migration names.
2. **Review SQL:** Always review generated SQL before applying.
3. **Test locally:** Verify migrations work on a copy of production data.
4. **Backup first:** Always backup before running migrations in production.

5. **Idempotent scripts:** Ensure migrations can be re-run if needed.

Previous
« Database Schema

Next
Seed Data »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Seed Data

The seed script populates the database with default data for local development and testing.

Running the Seed Script

```
cd backend
npx prisma db seed
```

Seed Contents

The seed script creates the following data:

Department

Field	Value
`name`	ICT
`code`	ICT

Users

Username	Password	Name	Role	Department
`admin`	`admin123`	System Administrator	`admin`	ICT
`itstaff`	`staff123`	IT Staff	`staff`	ICT

System Settings

Default settings are created if none exist:

Setting	Default Value
`organizationName`	KUCCPS IT Support
`organizationCode`	KUCCPS
`primaryColor`	#911414
`secondaryColor`	#d20001
`supportEmail`	itsupport@kuccps.ac.ke
`noreplyEmail`	noreply@kuccps.ac.ke
`emailFromName`	KUCCPS IT Support

Setting	Default Value
`ticketNumberPrefix`	TICK
`ticketNumberFormat`	YYYYMMDD-XXXX
`defaultPriority`	Medium
`defaultCategory`	General Issues
`defaultStatus`	Open
`maxFileSize`	10485760
`allowedFileTypes`	jpg,jpeg,png,gif,pdf,doc,docx,xls,xlsx,zip
`darkModeEnabled`	false
`contrastMode`	normal

Email Templates

Default email templates are created for:

- **ticketCreated:** Ticket creation confirmation
- **ticketUpdated:** Ticket status/priority change notification
- **ticketResolved:** Resolution notification and CSAT request

Seed Script Location

File: `backend/prisma/seed.js`

The seed script is defined in `backend/package.json`:

```
{
  "prisma": {
    "seed": "node ./prisma/seed.js"
  }
}
```

Customizing Seed Data

To customize seed data, edit `backend/prisma/seed.js`. You can add additional users, departments, or settings as needed for your environment.

Production Considerations

- **Never run `db seed` in production** — it will create duplicate records or overwrite existing data.
- **Change default passwords immediately** after seeding.
- **Use environment-specific seed scripts** for staging and production if needed.

Previous
« Migrations

Next
Prisma Usage »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Prisma Usage

This document describes how Prisma is used throughout the backend for database operations.

Prisma Client Setup

The Prisma Client is imported and instantiated in `backend/src/server.js`:

```
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
```

Common Query Patterns

Finding Records

```
// Find one by unique field
const user = await prisma.user.findUnique({
  where: { email: 'admin@example.com' }
});

// Find first matching record
```

```
const settings = await prisma.systemSettings.findFirst();

// Find many with filters
const openTickets = await prisma.ticket.findMany({
  where: { status: 'Open' },
  orderBy: { createdAt: 'desc' }
});
```

Creating Records

```
const ticket = await prisma.ticket.create({
  data: {
    subject: 'Printer not working',
    description: '...',
    status: 'Open',
    priority: 'Medium',
    requesterEmail: 'user@example.com',
    ticketNumber: await generateTicketNumber()
  }
});
```

Updating Records

```
const updated = await prisma.ticket.update({
  where: { id: ticketId },
  data: {
    status: 'Resolved',
    resolvedAt: new Date()
  }
});
```

Deleting Records

```
await prisma.ticket.delete({
  where: { id: ticketId }
});
```

Relations

Including Related Data

```
const ticket = await prisma.ticket.findUnique({
  where: { id: ticketId },
  include: {
    assignedTo: true,
    department: true,
    comments: {
      include: { user: true },
      orderBy: { createdAt: 'asc' }
    },
    attachments: true,
    history: {
      orderBy: { createdAt: 'desc' }
    }
  }
});
```

Creating Related Records

```
const comment = await prisma.comment.create({
```

```
  data: {
    content: 'Working on this issue',
    isInternal: false,
    ticketId: ticketId,
    userId: currentUser.id
  }
});
```

Transactions

For operations that must succeed or fail together:

```
const result = await prisma.$transaction(async (tx) => {
  const ticket = await tx.ticket.create({ data: ticketData });
  await tx.ticketHistory.create({
    data: {
      ticketId: ticket.id,
      field: 'status',
      oldValue: null,
      newValue: 'Open',
      changedBy: currentUser.id
    }
  });
  return ticket;
});
```

Raw Queries

When Prisma's query builder is insufficient:

```
const results = await prisma.$queryRaw`
  SELECT
    d.name,
    COUNT(t.id) as ticket_count
  FROM Department d
  LEFT JOIN Ticket t ON t.departmentId = d.id
  GROUP BY d.id, d.name
  ORDER BY ticket_count DESC
`;
```

Error Handling

Prisma throws specific error codes:

Code	Meaning
`P2002`	Unique constraint violation
`P2025`	Record not found
`P2003`	Foreign key constraint violation
`P2014`	Required relation missing

```
try {
  await prisma.user.create({ data: userData });
} catch (error) {
  if (error.code === 'P2002') {
    // Handle duplicate email/username
```

```
  }
}
```

Connection Management

Prisma manages connection pooling automatically. For serverless environments, consider using `@prisma/extension-pulse` or connection management tools.

Performance Tips

1. **Use `include` selectively** — only fetch related data when needed.
2. **Use `select` for large tables** — avoid fetching unnecessary columns.
3. **Add indexes** for frequently filtered fields.
4. **Use pagination** for large result sets.
5. **Use `$transaction`** for batch operations.

Previous
« Seed Data

Next
API Overview »

Docs

Getting Started

Architecture

API Reference

Community

GitHub

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

API Overview

All API endpoints are prefixed with `/api`. Responses are JSON.

Base URL

```
http://localhost:5000/api
```

Response Format

Standard response envelope:

```
{
  "success": true,
  "message": "Optional message",
  "data": {}
}
```

Authentication

JWT Bearer Token

Protected routes require a JWT token in the `Authorization` header:

```
Authorization: Bearer <jwt-token>
```

Public API Key

Public ticket endpoints require an API key:

```
X-API-Key: <PUBLIC_TICKET_API_KEY>
```

Or as a bearer token depending on server configuration.

Rate Limiting

Scope	Window	Max Requests
Global	15 minutes	600
Auth	15 minutes	30
Public tickets	15 minutes	30

Status Codes

Code	Meaning
200	Success
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
413	Payload Too Large
415	Unsupported Media Type
429	Too Many Requests
500	Server Error

Error Response Format

```
{
  "success": false,
  "message": "Error description"
```

```
}
```

Pagination

List endpoints support pagination via query parameters:

```
GET /api/tickets?page=1&limit=20
```

- `page`: Page number (default: 1)
- `limit`: Items per page (default: 20, max: 100)

Filtering

Many list endpoints support filtering:

```
GET /api/tickets?status=Open&priority=High&departmentId=uuid
```

Sorting

```
GET /api/tickets?sortBy=createdAt&sortOrder=desc
```

Previous
« Prisma Usage

Next
Authentication API »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Authentication API

Base path: `/api/auth`

Login

```
POST /api/auth/login
```

Request

```
{
  "username": "admin",
  "password": "admin123"
}
```

Response

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
```

```
    "id": "uuid",
    "email": "admin@example.com",
    "name": "System Administrator",
    "username": "admin",
    "role": "admin",
    "departmentId": "uuid",
    "department": {
      "id": "uuid",
      "name": "ICT"
    }
  }
}
```


Logout

```
POST /api/auth/logout
```

No request body required. Primarily for client-side token cleanup.

Profile

```
GET /api/auth/profile
```

Headers

```
Authorization: Bearer <jwt-token>
```

Response


```
{
  "success": true,
  "user": {
    "id": "uuid",
    "email": "admin@example.com",
    "name": "System Administrator",
    "username": "admin",
    "role": "admin",
    "departmentId": "uuid",
    "department": {
      "id": "uuid",
      "name": "ICT"
    }
  }
}
```


Previous

« API Overview

Next

Tickets API »

Architecture

API Reference

Community

GitHub

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Tickets API

Base path: `/api/tickets` (staff/admin) and `/api/public/tickets` (public)

Get Tickets

```
GET /api/tickets
```

Headers

```
Authorization: Bearer <jwt-token>
```

Query Parameters

Parameter	Type	Description
`page`	number	Page number (default: 1)
`limit`	number	Items per page (default: 20)

Parameter	Type	Description
`status`	string	Filter by status
`priority`	string	Filter by priority
`category`	string	Filter by category
`departmentId`	string	Filter by department
`assignedToId`	string	Filter by assignee
`search`	string	Search in subject/description
`sortBy`	string	Sort field
`sortOrder`	string	`asc` or `desc`

Response


```
{
  "success": true,
  "tickets": [
    {
      "id": "uuid",
      "ticketNumber": "TICK20260814-0001",
      "subject": "Cannot access printer",
      "status": "Open",
      "priority": "Medium",
      "category": "Hardware",
      "requesterEmail": "user@example.com",
```

```
      "department": { "id": "uuid", "name": "ICT" },
      "assignedTo": { "id": "uuid", "name": "John Doe" },
      "createdAt": "2026-08-14T09:00:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 42,
    "pages": 3
  }
}
```

Get Ticket

```
GET /api/tickets/:id
```

Response

```
{
  "success": true,
  "ticket": {
    "id": "uuid",
    "ticketNumber": "TICK20260814-0001",
    "subject": "Cannot access printer",
    "description": "The printer on the 3rd floor...",
    "status": "Open",
    "priority": "Medium",
    "category": "Hardware",
    "requesterEmail": "user@example.com",
    "requesterName": "John Doe",
```

```
    "slaDueAt": "2026-08-14T17:00:00.000Z",
    "slaBreached": false,
    "department": { "id": "uuid", "name": "ICT" },
    "assignedTo": { "id": "uuid", "name": "Jane Smith" },
    "createdBy": { "id": "uuid", "name": "John Doe" },
    "attachments": [
      { "id": "uuid", "filename": "error.png", "mimetype": "image/png" }
    ],
    "comments": [
      {
        "id": "uuid",
        "content": "Investigating...",
        "isInternal": false,
        "user": { "id": "uuid", "name": "Jane Smith" },
        "createdAt": "2026-08-14T09:30:00.000Z"
      }
    ],
    "history": [
      {
        "id": "uuid",
        "field": "status",
        "oldValue": "Open",
        "newValue": "In Progress",
        "createdAt": "2026-08-14T09:30:00.000Z"
      }
    ],
    "createdAt": "2026-08-14T09:00:00.000Z",
    "updatedAt": "2026-08-14T09:30:00.000Z"
  }
}
```


Update Ticket

```
PUT /api/tickets/:id
```

Request Body

```
{
  "status": "In Progress",
  "priority": "High",
  "assignedToId": "uuid",
  "departmentId": "uuid",
  "category": "Network",
  "slaDueAt": "2026-08-14T17:00:00.000Z"
}
```

Response

```
{
  "success": true,
  "message": "Ticket updated successfully",
  "ticket": { /* updated ticket object */ }
}
```

Escalate Ticket

```
POST /api/tickets/:id/escalate
```

Request Body

```
{
  "escalateToId": "uuid",
  "reason": "Requires senior network engineer"
}
```

Add Comment

```
POST /api/tickets/:id/comments
```

Request Body

```
{
  "content": "Working on this issue",
  "isInternal": false
}
```

Upload Attachment

```
POST /api/tickets/:id/attachments
Content-Type: multipart/form-data
```

Form Data

Field	Type	Description
`file`	File	The file to upload

Response

```
{
  "success": true,
  "message": "Attachment uploaded",
  "attachment": {
    "id": "uuid",
    "filename": "screenshot.png",
    "mimetype": "image/png",
    "size": 204800,
    "createdAt": "2026-08-14T09:00:00.000Z"
  }
}
```

Create Public Ticket

```
POST /api/public/tickets
```

Headers

```
X-API-Key: <PUBLIC_TICKET_API_KEY>
Content-Type: application/json
```

Request Body

```
{
  "subject": "Cannot access printer",
  "description": "The printer on the 3rd floor is not responding.",
  "category": "Hardware",
  "priority": "Medium",
  "requesterEmail": "user@example.com",
  "requesterName": "John Doe"
}
```

Response

```
{
  "success": true,
  "message": "Ticket created successfully",
  "ticket": {
    "id": "uuid",
    "ticketNumber": "TICK20260814-0001",
    "subject": "Cannot access printer",
    "status": "Open",
    "createdAt": "2026-08-14T09:00:00.000Z"
  }
}
```

Previous
« Authentication API

Next
Users API »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Users API

Base path: `/api/users`

Get Users

```
GET /api/users
```

Headers

```
Authorization: Bearer <jwt-token>
```

Response

```
{
  "success": true,
  "users": [
    {
      "id": "uuid",
      "email": "admin@example.com",
      "name": "System Administrator",
```

```
      "username": "admin",
      "role": "admin",
      "ldapSync": false,
      "departmentId": "uuid",
      "department": {
        "id": "uuid",
        "name": "ICT"
      },
      "createdAt": "2026-08-14T09:00:00.000Z"
    }
  ]
}
```


Create User

```
POST /api/users
```

Headers

```
Authorization: Bearer <jwt-token>
```

Request Body


```
{
  "email": "newuser@example.com",
  "name": "New User",
  "username": "newuser",
  "password": "securePassword123",
  "role": "staff",
```

```
  "departmentId": "uuid"
}
```

Response

```
{
  "success": true,
  "message": "User created successfully",
  "user": { /* user object */ }
}
```

Update User

```
PUT /api/users/:id
```

Request Body

```
{
  "name": "Updated Name",
  "role": "admin",
  "departmentId": "uuid",
  "password": "newPassword123"
}
```

> Note: Password is optional. If provided, it will be hashed and updated.

Delete User

```
DELETE /api/users/:id
```

Response

```
{
  "success": true,
  "message": "User deleted successfully"
}
```

Sync from Active Directory

```
POST /api/users/sync/ad
```

Headers

```
Authorization: Bearer <jwt-token>
```

> Admin only.

Response

```
{
```

```
  "success": true,
  "message": "Users synced from Active Directory",
  "synced": 15
}
```

Previous
« Tickets API

Next
Departments API »

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Departments API

Base path: `/api/departments`

Get Departments

```
GET /api/departments
```

Headers

```
Authorization: Bearer <jwt-token>
```

Response

```
{
  "success": true,
  "departments": [
    {
      "id": "uuid",
      "name": "ICT",
      "code": "ICT",
```

```
      "createdAt": "2026-08-14T09:00:00.000Z",
      "updatedAt": "2026-08-14T09:00:00.000Z"
    }
  ]
}
```

Get Department

```
GET /api/departments/:id
```

Response

```
{
  "success": true,
  "department": {
    "id": "uuid",
    "name": "ICT",
    "code": "ICT",
    "users": [
      { "id": "uuid", "name": "John Doe", "role": "staff" }
    ],
    "slaPolicies": [
      { "id": "uuid", "name": "Standard SLA", "isActive": true }
    ],
    "createdAt": "2026-08-14T09:00:00.000Z",
    "updatedAt": "2026-08-14T09:00:00.000Z"
  }
}
```

Create Department

```
POST /api/departments
```

Request Body

```
{
  "name": "Human Resources",
  "code": "HR"
}
```

Update Department

```
PUT /api/departments/:id
```

Request Body

```
{
  "name": "Human Resources",
  "code": "HR"
}
```

Delete Department

```
DELETE /api/departments/:id
```

Response

```
{
  "success": true,
  "message": "Department deleted successfully"
}
```

Sync from Active Directory

```
POST /api/departments/sync/ad
```

> Admin only.

Response

```
{
  "success": true,
  "message": "Directorates synced from Active Directory",
  "synced": 8
}
```

Previous
« Users API

Next
Settings API »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Settings API

Base path: `/api/settings`

Get Settings

```
GET /api/settings
```

Headers

```
Authorization: Bearer <jwt-token>
```

> Admin only.

Response

```
{
  "success": true,
  "settings": {
    "organizationName": "KUCCPS IT Support",
    "organizationCode": "KUCCPS",
```

```
    "primaryColor": "#911414",
    "secondaryColor": "#d20001",
    "supportEmail": "itsupport@kuccps.ac.ke",
    "smtpHost": "smtp.gmail.com",
    "smtpPort": 587,
    "imapHost": "imap.gmail.com",
    "imapPort": 993,
    "ldapEnabled": false,
    "darkModeEnabled": false,
    "contrastMode": "normal"
  }
}
```


Update Settings

```
PUT /api/settings
```

Request Body


```
{
  "organizationName": "KUCCPS IT Support",
  "primaryColor": "#911414",
  "secondaryColor": "#d20001",
  "supportEmail": "itsupport@kuccps.ac.ke",
  "smtpHost": "smtp.gmail.com",
  "smtpPort": 587,
  "smtpUser": "user@gmail.com",
  "smtpPassword": "app-password",
  "imapHost": "imap.gmail.com",
  "imapPort": 993,
  "imapUser": "user@gmail.com",
```

```
  "imapPassword": "app-password",
  "ldapEnabled": true,
  "ldapUrl": "ldaps://ldap.example.ac.ke:636"
}
```

Get Public Settings

```
GET /api/settings/public
```

No authentication required. Returns public branding settings:

```
{
  "success": true,
  "settings": {
    "organizationName": "KUCCPS IT Support",
    "primaryColor": "#911414",
    "secondaryColor": "#d20001",
    "logoUrl": null,
    "faviconUrl": null,
    "darkModeEnabled": false,
    "contrastMode": "normal"
  }
}
```

Email Templates

List Templates

```
GET /api/settings/email-templates
```

Get Template

```
GET /api/settings/email-templates/ticketCreated
```

Create Template

```
POST /api/settings/email-templates
```

```
{
  "type": "ticketCreated",
  "subject": "Ticket Created: {{ticketNumber}}",
  "html": "<p>Your ticket {{ticketNumber}} has been created.</p>"
}
```

Update Template

```
PUT /api/settings/email-templates/ticketCreated
```

Delete Template

```
DELETE /api/settings/email-templates/ticketCreated
```

Reset Templates

```
POST /api/settings/email-templates/reset
```

Resets all templates to their default values.

Previous
« Departments API

Next
Knowledge Base API »

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Knowledge Base API

Base path: `/api/knowledge-base`

Public Articles

List Public Articles

```
GET /api/knowledge-base/public/articles
```

Response


```
{
  "success": true,
  "articles": [
    {
      "id": "uuid",
      "title": "How to reset your password",
      "slug": "how-to-reset-password",
      "excerpt": "Step-by-step guide...",
      "category": "Account",
      "status": "published",
      "views": 142,
```

```
      "helpful": 120,
      "notHelpful": 8,
      "createdAt": "2026-08-14T09:00:00.000Z"
    }
  ]
}
```


Get Public Article

```
GET /api/knowledge-base/public/articles/:idOrSlug
```

Staff/Admin Articles

List Articles

```
GET /api/knowledge-base/articles
```

Get Article

```
GET /api/knowledge-base/articles/:idOrSlug
```

Create Article

```
POST /api/knowledge-base/articles
```

```
{
  "title": "How to reset your password",
  "slug": "how-to-reset-password",
  "excerpt": "Step-by-step guide...",
  "content": "<p>Full article content...</p>",
  "category": "Account",
  "tags": "password,reset,account",
  "status": "published"
}
```

Update Article

```
PUT /api/knowledge-base/articles/:id
```

Delete Article

```
DELETE /api/knowledge-base/articles/:id
```

Rate Article

```
POST /api/knowledge-base/articles/:id/rate
```

```
{
  "helpful": true
}
```

Upload Material

```
POST /api/knowledge-base/materials/upload
Content-Type: multipart/form-data
```

Insights

```
GET /api/knowledge-base/insights
```

```
{
  "success": true,
  "insights": {
    "totalArticles": 25,
    "publishedArticles": 20,
    "totalViews": 1520,
    "topArticles": [
      { "id": "uuid", "title": "...", "views": 142 }
    ]
  }
}
```

Suggestions

```
GET /api/knowledge-base/suggestions/tickets/:ticketId
```

Returns KB articles suggested for a specific ticket based on category and content.

Previous
« Settings API

Next
Status Issues API »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Status Issues API

Base path: `/api/status-issues`

Public Status Issues

List Public Issues

```
GET /api/status-issues/public
```

Response


```
{
  "success": true,
  "issues": [
    {
      "id": "uuid",
      "title": "Email Service Outage",
      "description": "Email delivery is delayed...",
      "status": "Investigating",
      "severity": "Error",
      "systems": "Email",
      "isPublic": true,
```

```
      "startedAt": "2026-08-14T08:00:00.000Z",
      "updates": [
        {
          "id": "uuid",
          "message": "We are investigating the issue.",
          "status": "Investigating",
          "isPublic": true,
          "createdAt": "2026-08-14T08:30:00.000Z"
        }
      ]
    }
  ]
}
```

Staff/Admin Issues

List Issues

```
GET /api/status-issues
```

Create Issue

```
POST /api/status-issues
```

```
{
  "title": "Email Service Outage",
  "description": "Email delivery is delayed for all users.",
  "status": "Investigating",
  "severity": "Error",
```

```
  "systems": "Email, SMTP",
  "isPublic": true
}
```

Update Issue

```
PUT /api/status-issues/:id
```

```
{
  "status": "Resolved",
  "severity": "Error",
  "isPublic": true
}
```

Add Update

```
POST /api/status-issues/:id/updates
```

```
{
  "message": "Service has been restored.",
  "status": "Resolved",
  "isPublic": true
}
```

Previous
« Knowledge Base API

Next
Operations API »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Operations API

Base path: `/api/operations`

Dashboard Analytics

```
GET /api/operations/dashboard
```

Headers

```
Authorization: Bearer <jwt-token>
```

Response

```
{
  "success": true,
  "statistics": {
    "totalTickets": 150,
    "openTickets": 45,
    "inProgressTickets": 30,
    "resolvedTickets": 70,
```

```
    "closedTickets": 5,
    "criticalTickets": 3,
    "overdueTickets": 8,
    "avgResponseTime": 120,
    "avgResolutionTime": 480,
    "slaComplianceRate": 0.92,
    "ticketsByCategory": {
      "Hardware": 30,
      "Software": 50,
      "Network": 25,
      "Access": 20,
      "General Issues": 25
    },
    "ticketsByPriority": {
      "Low": 40,
      "Medium": 70,
      "High": 30,
      "Critical": 10
    },
    "ticketsByStatus": {
      "Open": 45,
      "In Progress": 30,
      "Resolved": 70,
      "Closed": 5
    },
    "ticketsByHour": [
      { "hour": "09:00", "count": 12 },
      { "hour": "10:00", "count": 18 }
    ],
    "departmentStats": [
      { "name": "ICT", "count": 60 },
      { "name": "HR", "count": 30 }
    ],
    "topPerformers": [
      { "name": "Jane Smith", "resolved": 25, "avgTime": 180 }
    ]
  }
```

```
}
```

SLA Policies

List Policies

```
GET /api/operations/sla-policies
```

Create Policy

```
POST /api/operations/sla-policies
```

```
{
  "name": "Standard SLA",
  "priority": "Medium",
  "category": "Software",
  "departmentId": "uuid",
  "responseMinutes": 240,
  "resolutionMinutes": 1440,
  "isActive": true
}
```

Update Policy

```
PUT /api/operations/sla-policies/:id
```

CSAT Response

```
POST /api/operations/csat
```

Headers

```
Authorization: Bearer <jwt-token>
```

Request Body

```
{
  "ticketId": "uuid",
  "rating": 5,
  "comment": "Excellent support!"
}
```

Response

```
{
  "success": true,
  "message": "CSAT response submitted",
  "csat": { /* csat object */ }
}
```

Previous
« Status Issues API

Next
Notifications API »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Notifications API

Base path: `/api/notifications`

All notification endpoints require JWT authentication.

Get Notifications

```
GET /api/notifications
```

Response


```
{
  "success": true,
  "notifications": [
    {
      "id": "uuid",
      "type": "sla_breach",
      "title": "SLA Breach Alert",
      "message": "Ticket TICK20260814-0001 has breached its SLA.",
      "isRead": false,
      "ticketId": "uuid",
      "ticketNumber": "TICK20260814-0001",
      "createdAt": "2026-08-14T09:00:00.000Z"
```

```
    }
  ]
}
```

Mark All as Read

```
PUT /api/notifications/read/all
```

Response

```
{
  "success": true,
  "message": "All notifications marked as read"
}
```

Mark One as Read

```
PUT /api/notifications/:id/read
```

Dismiss Notification

```
DELETE /api/notifications/:id
```

Get Preferences

```
GET /api/notifications/preferences
```

Response

```
{
  "success": true,
  "preferences": {
    "emailOnAssignment": true,
    "emailOnStatusChange": true,
    "emailOnNewComment": true,
    "dailyDigest": false,
    "escalationAlerts": true
  }
}
```

Update Preferences

```
PUT /api/notifications/preferences
```

```
{
  "emailOnAssignment": true,
  "emailOnStatusChange": false,
  "emailOnNewComment": true,
  "dailyDigest": true,
  "escalationAlerts": true
```

```
}
```

Send Test Email

```
POST /api/notifications/test
```

Sends a test notification email to the current user's email address.

```
{
  "success": true,
  "message": "Test email sent"
}
```

Previous
« Operations API

Next
Forms API »

Architecture

API Reference

Community

GitHub

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Forms API

Base path: `/api/forms`

Public Forms

List Active Forms

```
GET /api/forms/public
```

No authentication required. Returns only active forms.

Response


```
{
  "success": true,
  "forms": [
    {
      "id": "uuid",
      "name": "Hardware Request",
      "category": "Hardware",
      "description": "Request new hardware equipment",
      "isActive": true,
```

```
      "fields": [
        {
          "id": "uuid",
          "label": "Equipment Type",
          "key": "equipmentType",
          "type": "select",
          "required": true,
          "options": ["Laptop", "Monitor", "Keyboard", "Mouse"]
        },
        {
          "id": "uuid",
          "label": "Justification",
          "key": "justification",
          "type": "textarea",
          "required": true
        }
      ]
    }
  ]
}
```

Staff/Admin Forms

List Forms

```
GET /api/forms
```

Create Form

```
POST /api/forms
```

```
{
  "name": "Hardware Request",
  "category": "Hardware",
  "description": "Request new hardware equipment",
  "isActive": true,
  "fields": [
    {
      "label": "Equipment Type",
      "key": "equipmentType",
      "type": "select",
      "required": true,
      "options": ["Laptop", "Monitor", "Keyboard", "Mouse"],
      "sortOrder": 0
    },
    {
      "label": "Justification",
      "key": "justification",
      "type": "textarea",
      "required": true,
      "sortOrder": 1
    }
  ]
}
```

Update Form

```
PUT /api/forms/:id
```

Delete Form

```
DELETE /api/forms/:id
```

Form Field Types

Type	HTML Input	Validation
`text`	`<input type="text">`	Required/optional
`textarea`	`<textarea>`	Required/optional
`email`	`<input type="email">`	Email format
`select`	`<select>`	Must match one of `options`
`multiselect`	Multiple checkboxes	Must match `options`
`date`	`<input type="date">`	Date format
`number`	`<input type="number">`	Numeric
`file`	`<input type="file">`	File type/size limits

Conditional Fields

Fields can have `conditions` to control visibility:

```
{
  "conditions": {
    "field": "equipmentType",
    "operator": "equals",
    "value": "Laptop",
    "showFields": ["ram", "storage"]
  }
}
```

Previous
« Notifications API

Next
Views API »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Views API

Base path: `/api/views`

Saved views allow users to save and reuse ticket filter configurations.

Get Views

```
GET /api/views
```

Headers

```
Authorization: Bearer <jwt-token>
```

Response

```
{
  "success": true,
  "views": [
    {
      "id": "uuid",
```

```
      "name": "My Open Tickets",
      "entity": "tickets",
      "filters": {
        "status": "Open",
        "assignedToId": "current-user-uuid"
      },
      "columns": ["ticketNumber", "subject", "status", "priority", "createdAt"],
      "sortBy": "createdAt",
      "sortOrder": "desc",
      "visibility": "private",
      "role": null,
      "ownerId": "uuid",
      "createdAt": "2026-08-14T09:00:00.000Z",
      "updatedAt": "2026-08-14T09:00:00.000Z"
    }
  ]
}
```

Create View

```
POST /api/views
```

```
{
  "name": "Critical Hardware Issues",
  "entity": "tickets",
  "filters": {
    "priority": "Critical",
    "category": "Hardware"
  },
  "columns": ["ticketNumber", "subject", "status", "assignedTo", "createdAt"],
  "sortBy": "createdAt",
  "sortOrder": "desc",
```

```
  "visibility": "private"
}
```

Visibility Options

Value	Scope
`private`	Owner only
`shared`	All staff
`global`	All users (including public)

Update View

```
PUT /api/views/:id
```

```
{
  "name": "Updated View Name",
  "filters": {
    "status": "Open",
    "priority": "High"
  },
  "sortBy": "priority",
  "sortOrder": "asc"
}
```

Delete View

```
DELETE /api/views/:id
```

View Structure

```
{
  "id": "uuid",
  "name": "string",
  "entity": "string",
  "filters": {
    "status": "string | string[]",
    "priority": "string | string[]",
    "category": "string | string[]",
    "departmentId": "string | string[]",
    "assignedToId": "string | string[]",
    "createdById": "string | string[]",
    "dateFrom": "ISO date string",
    "dateTo": "ISO date string",
    "search": "string"
  },
  "columns": ["string"],
  "sortBy": "string",
  "sortOrder": "asc | desc",
  "visibility": "private | shared | global",
  "role": "string | null",
  "ownerId": "string | null"
}
```

Previous
« Forms API

Next
Google Workspace Add-on Overview »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Google Workspace Add-on Overview

The Google Workspace Add-on allows KUCCPS staff to create support tickets directly from Gmail.

Quick Facts

Aspect	Detail
Platform	Google Apps Script
Type	Gmail contextual add-on
Integration	Backend public ticket API
Auth	API key (`PUBLIC_TICKET_API_KEY`)

What the Add-on Does

- Provides a compose trigger in Gmail to create tickets
- Pre-fills ticket details from email context
- Submits tickets to the backend public API
- Displays the generated ticket number to the user

Files

File	Purpose
`Code.gs`	Main Apps Script code
`appsscript.json`	Add-on manifest and configuration

Documentation Sections

- Setup — Creating and configuring the Apps Script project
- Deployment — Deploying to production
- API Integration — Communicating with the backend

Previous
« Views API

Next

Add-on Setup »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Add-on Setup

This guide covers setting up the Google Workspace Add-on for development and testing.

Prerequisites

- A Google Workspace account with access to Apps Script
- The backend deployed and accessible from the internet
- A `PUBLIC_TICKET_API_KEY` configured on the backend

Step 1: Create Apps Script Project

1. Go to script.google.com
2. Click **New Project**
3. Rename the project to `KUCCPS IT Support Add-on`

Step 2: Paste Code

1. Open `Code.gs` in the Apps Script editor

2. Paste the contents of `google-addon/Code.gs`
3. Open `appsscript.json` and paste the contents of `google-addon/appsscript.json`

Step 3: Configure Script Properties

1. In the Apps Script IDE, click **Project Settings** (gear icon)
2. Scroll to **Script Properties**
3. Add the following properties:

Property	Value	Description
`API_URL`	`http://localhost:5000/api`	Backend API URL (use ngrok/tunnel for local testing)
`BACKEND_API_KEY`	Your public API key	Authentication key
`SUPPORT_EMAIL`	`itsupport@kuccps.ac.ke`	Support email address

Step 4: Run Setup Function

Run `setupProductionProperties` from the Apps Script editor to populate properties if they are empty:

```
function setupProductionProperties() {
  const properties = PropertiesService.getScriptProperties();
  properties.setProperty('API_URL', 'https://api.example.ac.ke/api');
  properties.setProperty('BACKEND_API_KEY', 'your-api-key-here');
```

```
  properties.setProperty('SUPPORT_EMAIL', 'itsupport@kuccps.ac.ke');
}
```

Step 5: Test Deployment

1. Click **Deploy > Test deployments**
2. Select **Install** to test as yourself
3. Open Gmail and look for the add-on icon in the compose window or email thread

Step 6: Create Production Deployment

1. Click **Deploy > New deployment**
2. Select **Add-on** as the deployment type
3. Configure deployment settings
4. Click **Install** to install for your domain

Local Testing with Tunnels

For local development, use a tunnel service like `ngrok` or `cloudflared` to expose your local backend:

```
# Using ngrok
ngrok http 5000

# Using cloudflared
```

```
cloudflared tunnel --url http://localhost:5000
```

Update `API_URL` in Apps Script properties to the tunnel URL.

> **Warning:** Do not use temporary tunnel URLs for production. Use a named Cloudflare Tunnel or official HTTPS domain.

Debugging

- Use **View > Logs** in Apps Script to see console output.
- Use `Logger.log()` statements in `Code.gs` for debugging.
- Check backend logs for API request details.

Previous
« Google Workspace Add-on Overview

Next
Add-on Deployment »

API Reference

Community

GitHub

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Add-on Deployment

This guide covers deploying the Google Workspace Add-on to production.

Pre-Deployment Checklist

- [ ] Backend is deployed and accessible via HTTPS
- [ ] `PUBLIC_TICKET_API_KEY` is configured in production backend
- [ ] `REQUIRE_PUBLIC_TICKET_API_KEY=true` is set
- [ ] Google Workspace Add-on account is ready
- [ ] API_URL points to production backend
- [ ] Domain installation is configured

Step 1: Update API URL

1. Open Apps Script project properties
2. Update `API_URL` to production backend URL:

```
https://api.kuccps.ac.ke/api
```

Step 2: Update API Key

Update `BACKEND_API_KEY` to the production API key.

Step 3: Create New Deployment

1. In Apps Script IDE, click **Deploy > Manage deployments**
2. Click **New deployment**
3. Select **Add-on** type
4. Enter a version description (e.g., "Production v1.0")
5. Click **Deploy**

Step 4: Domain Install

1. In Google Admin Console, go to **Apps > Google Workspace > Settings for Apps**
2. Find the add-on and configure **Installation access**
3. Set to **Install for everyone** or specific organizational units
4. Save changes

Step 5: Verify Installation

1. Open Gmail as a test user
2. Look for the add-on icon in the compose window
3. Create a test ticket

4. Verify it appears in the backend

Updating the Add-on

After every `Code.gs` change:

1. Click **Deploy > Manage deployments**
2. Click **Edit** on the current deployment
3. Select **New version**
4. Click **Deploy**

Troubleshooting Deployment

Issue	Solution
Add-on not visible	Ensure it is deployed as an add-on, not a web app
401 errors	Check `BACKEND_API_KEY` matches backend config
CORS errors	Ensure `FRONTEND_URL` or `CORS_ORIGINS` includes the Apps Script domain
Network errors	Verify `API_URL` is accessible from the public internet

Previous
« Add-on Setup

Next
Add-on API Integration »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Add-on API Integration

This document describes how the Google Workspace Add-on communicates with the backend API.

Endpoint Reference

Method	Endpoint	Purpose
`POST`	`/api/public/tickets`	Create a new ticket
`POST`	`/api/tickets/public`	Alias for Google Add-on compatibility

Authentication

All requests must include the `X-API-Key` header:

```
X-API-Key: <PUBLIC_TICKET_API_KEY>
Content-Type: application/json
```

Create Ticket Request

```
function createTicket(formData) {
  const apiUrl = PropertiesService.getScriptProperties().getProperty('API_URL');
  const apiKey =
PropertiesService.getScriptProperties().getProperty('BACKEND_API_KEY');

  const payload = {
    subject: formData.subject,
    description: formData.description,
    category: formData.category,
    priority: formData.priority,
    requesterEmail: Session.getActiveUser().getEmail(),
    requesterName: Session.getActiveUser().getDisplayName()
  };

  const options = {
    method: 'post',
    contentType: 'application/json',
    headers: {
      'X-API-Key': apiKey
    },
    payload: JSON.stringify(payload),
    muteHttpExceptions: true
  };

  const response = UrlFetchApp.fetch(`${apiUrl}/public/tickets`, options);
  return JSON.parse(response.getContentText());
}
```

Create Ticket Response

Success:

```
{
  "success": true,
  "message": "Ticket created successfully",
  "ticket": {
    "id": "uuid",
    "ticketNumber": "TICK20260814-0001",
    "subject": "Cannot access printer",
    "status": "Open",
    "createdAt": "2026-08-14T09:00:00.000Z"
  }
}
```

Error:

```
{
  "success": false,
  "message": "Invalid API key"
}
```

Error Handling

```
function handleResponse(response) {
  const result = JSON.parse(response.getContentText());

  if (result.success) {
    return {
      ticketNumber: result.ticket.ticketNumber,
      message: result.message
    };
```

```
  } else {
    throw new Error(result.message || 'Unknown error');
  }
}
```

Rate Limits

The public ticket endpoint is rate-limited:

- **30 requests per 15 minutes** per IP address

Security Considerations

1. **Never log the API key** in client-side code.
2. **Validate inputs** before sending to the API.
3. **Handle errors gracefully** in the add-on UI.
4. **Use HTTPS** for the backend URL in production.

Previous
« Add-on Deployment

Next
Deployment Overview »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Deployment Overview

This section covers deploying the KUCCPS IT Ticketing System to production.

Architecture

Deployment Options

Component	Recommended	Alternatives
Frontend	Vercel, Netlify, or static VPS	Cloudflare Pages, AWS S3 + CloudFront
Backend	Railway, Render, or VPS	AWS EC2, Google Cloud Run, Docker
Database	Supabase, Neon, or managed PostgreSQL	AWS RDS, Google Cloud SQL

Component	Recommended	Alternatives
Reverse Proxy	Nginx on same VPS as backend	Caddy, Cloudflare Tunnel

Pre-Deployment Checklist

See Production Checklist for the full list.

Key items:

1. Rotate `JWT_SECRET` and `PUBLIC_TICKET_API_KEY`
2. Set `NODE_ENV=production`
3. Configure explicit CORS origins (HTTPS)
4. Set `REQUIRE_PUBLIC_TICKET_API_KEY=true`
5. Configure SMTP and test email delivery
6. Enable LDAPS for AD integration
7. Backup existing data
8. Test all features in staging

Previous
« Add-on API Integration

Next

Backend Deployment »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Backend Deployment

This guide covers deploying the Node.js/Express backend to production.

Platform Options

Platform	Pros	Cons
Railway	Easy deployment, PostgreSQL included, auto-scaling	Higher cost at scale
Render	Free tier, easy setup	Cold starts on free tier
VPS (Ubuntu)	Full control, cost-effective	Manual setup and maintenance
Docker	Consistent environment, portable	Requires container knowledge
AWS EC2 / GCP	Scalable, enterprise-grade	Complex setup

Environment Variables

Set all environment variables in your deployment platform's dashboard:

```
NODE_ENV=production
DATABASE_URL=postgresql://user:pass@host:5432/kuccps_ticketing
JWT_SECRET=<strong-secret-here>
PUBLIC_TICKET_API_KEY=<strong-key-here>
REQUIRE_PUBLIC_TICKET_API_KEY=true
FRONTEND_URL=https://ticketing.kuccps.ac.ke
CORS_ORIGINS=https://ticketing.kuccps.ac.ke
EMAIL_SERVICE=gmail
EMAIL_USER=itsupport@kuccps.ac.ke
EMAIL_PASSWORD=<app-password>
SUPPORT_EMAIL=itsupport@kuccps.ac.ke
IMAP_HOST=imap.gmail.com
IMAP_PORT=993
IMAP_USER=itsupport@kuccps.ac.ke
IMAP_PASSWORD=<app-password>
```

Railway Deployment

1. Create a new project on Railway
2. Connect your GitHub repository
3. Select the `backend` directory as the root
4. Add PostgreSQL plugin
5. Set environment variables
6. Deploy

Railway automatically runs `npm install` and `npm start`.

Docker Deployment

Create a `Dockerfile` in `backend/`:

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npx prisma generate
EXPOSE 5000
CMD ["npm", "start"]
```

Build and run:

```
docker build -t kuccps-backend ./backend
docker run -p 5000:5000 --env-file backend/.env kuccps-backend
```

PM2 (VPS)

For process management on a VPS:

```
npm install -g pm2
cd backend
pm2 start src/server.js --name kuccps-backend
pm2 save
```

```
pm2 startup
```

Health Checks

Configure your platform to check `/health`:

```
curl https://api.kuccps.ac.ke/health
```

Expected response:

```
{
  "status": "OK",
  "message": "KUCCPS Ticketing System API is running"
}
```

Logs

In production, logs should include request IDs for tracing:

```
console.error('Unhandled request error:', { requestId: req.id, message: err.message });
```

Redirect logs to a file or logging service (e.g., Papertrail, Logflare).

Previous
« Deployment Overview

Next
Frontend Deployment »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Frontend Deployment

This guide covers deploying the React frontend to production.

Build Process

```
cd frontend
npm install
npm run build
```

The build output is in `frontend/dist/`.

Deployment Options

Platform	Pros	Cons
Vercel	Zero-config, preview deployments, edge network	Requires Git integration
Netlify	Drag-and-drop, form handling, edge functions	Similar to Vercel

Platform	Pros	Cons
Cloudflare Pages	Free tier, global CDN, Pages Functions	Smaller ecosystem
Nginx (VPS)	Full control, cost-effective	Manual setup

Vercel Deployment

1. Push your code to GitHub
2. Create a new project on Vercel
3. Import the repository
4. Set the **Root Directory** to `frontend/`
5. Add environment variable:

   ```
   VITE_API_URL=https://api.kuccps.ac.ke/api
   ```

6. Click **Deploy**

Netlify Deployment

1. Build the project: `npm run build`
2. Drag the `dist/` folder to Netlify
3. Or connect Git repository and configure:
   - Build command: `npm run build`

- Publish directory: `dist`

4. Set environment variable `VITE_API_URL`

Nginx (VPS)

```
server {
    listen 80;
    server_name ticketing.kuccps.ac.ke;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name ticketing.kuccps.ac.ke;

    ssl_certificate /etc/letsencrypt/live/ticketing.kuccps.ac.ke/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/ticketing.kuccps.ac.ke/privkey.pem;

    root /var/www/ticketing/dist;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api {
        proxy_pass http://localhost:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Environment Variables in Production

Set `VITE_API_URL` to the production backend URL:

```
VITE_API_URL=https://api.kuccps.ac.ke/api
```

> **Important:** Variables prefixed with `VITE_` are exposed to the browser. Never store secrets in frontend environment variables.

SPA Routing

The frontend is a single-page application. Configure your server to serve `index.html` for all routes:

```
location / {
    try_files $uri $uri/ /index.html;
}
```

This ensures client-side routing works correctly.

Caching

Static assets should be cached aggressively:

```
location ~* \.(js|css|png|jpg|svg|ico|woff2?)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
```

```
}
```

`index.html` should not be cached to ensure updates are served immediately.

Previous
« Backend Deployment

Next
Database Deployment »

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Database Deployment

This guide covers deploying and managing the PostgreSQL database in production.

Managed PostgreSQL (Recommended)

Provider	Free Tier	Notes
Supabase	Yes	PostgreSQL with REST API, easy setup
Neon	Yes	Serverless PostgreSQL, branching
Railway	$5/month	Integrated with Railway backend
AWS RDS	No	Enterprise-grade, managed backups

Self-Hosted PostgreSQL

Installation

```
# Ubuntu/Debian
sudo apt update
sudo apt install postgresql postgresql-contrib

# Start service
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

Create Database

```
sudo -u postgres psql
CREATE DATABASE kuccps_ticketing;
CREATE USER kuccps_user WITH PASSWORD 'secure-password';
GRANT ALL PRIVILEGES ON DATABASE kuccps_ticketing TO kuccps_user;
\q
```

Migrations

Run migrations on the production database:

```
cd backend
npx prisma migrate deploy
npx prisma generate
```

> **Never run `prisma migrate dev` in production.**

Backups

Automated Backups

Configure automated PostgreSQL backups:

```
# Daily backup script
0 2 * * * pg_dump -U kuccps_user kuccps_ticketing | gzip > /backups/kuccps_$(date
+\%Y\%m\%d).sql.gz
```

pg_dump

```
# Full backup
pg_dump -U kuccps_user -d kuccps_ticketing > backup.sql

# Compressed backup
pg_dump -U kuccps_user -d kuccps_ticketing | gzip > backup.sql.gz

# Specific tables
pg_dump -U kuccps_user -d kuccps_ticketing -t Ticket -t User > tables_backup.sql
```

Restore

```
# Restore from backup
psql -U kuccps_user -d kuccps_ticketing < backup.sql

# Restore compressed
gunzip -c backup.sql.gz | psql -U kuccps_user -d kuccps_ticketing
```

Backup Best Practices

1. **Daily automated backups** with at least 7-day retention
2. **Test restore procedures** monthly
3. **Store backups offsite** (e.g., S3, Google Cloud Storage)
4. **Back up file uploads** separately from database
5. **Document restore procedures** for disaster recovery

Connection Pooling

For high-traffic deployments, use PgBouncer:

```
# Install PgBouncer
sudo apt install pgbouncer

# Configure pool_mode = transaction
# Update DATABASE_URL to use PgBouncer port
```

Monitoring

Monitor database health:

- Connection count
- Query performance
- Disk usage
- Replication lag (if using replicas)

Tools:

- `pg_stat_activity`
- `pg_stat_user_tables`
- `pg_size_pretty(pg_database_size('kuccps_ticketing'))`

Previous
« Frontend Deployment

Next
Reverse Proxy »

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Reverse Proxy

A reverse proxy sits between the internet and your application, handling HTTPS, routing, and security.

Nginx Configuration

Full Configuration

```
# Rate limiting
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
limit_req_zone $binary_remote_addr zone=auth:10m rate=5r/s;

# HTTP to HTTPS redirect
server {
    listen 80;
    server_name ticketing.kuccps.ac.ke;
    return 301 https://$server_name$request_uri;
}

# HTTPS server
server {
    listen 443 ssl http2;
    server_name ticketing.kuccps.ac.ke;
```

```
    # SSL Configuration
    ssl_certificate /etc/letsencrypt/live/ticketing.kuccps.ac.ke/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/ticketing.kuccps.ac.ke/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-RSA-AES256-GCM-SHA512:DHE-RSA-AES256-GCM-SHA512;
    ssl_prefer_server_ciphers off;

    # Frontend
    root /var/www/ticketing/dist;
    index index.html;

    # Security headers
    add_header X-Frame-Options "DENY" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Permissions-Policy "camera=(), microphone=(), geolocation=()" always;

    # Static assets caching
    location ~* \.(js|css|png|jpg|jpeg|gif|svg|ico|woff|woff2|ttf|eot)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
        try_files $uri =404;
    }

    # SPA routing - serve index.html for all non-API routes
    location / {
        try_files $uri $uri/ /index.html;
        expires -1;
        add_header Cache-Control "no-store";
    }

    # API proxy
    location /api {
        limit_req zone=api burst=20 nodelay;

        proxy_pass http://localhost:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
```

```
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Request-Id $request_id;

        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }

    # Auth endpoints with stricter rate limiting
    location /api/auth {
        limit_req zone=auth burst=5 nodelay;

        proxy_pass http://localhost:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Health check
    location /health {
        proxy_pass http://localhost:5000/health;
        access_log off;
    }
}
```

SSL Certificates

Let's Encrypt with Certbot

```
# Install Certbot
sudo apt install certbot python3-certbot-nginx
```

```
# Obtain certificate
sudo certbot --nginx -d ticketing.kuccps.ac.ke

# Auto-renewal
sudo certbot renew --dry-run
```

Cloudflare Tunnel (Alternative)

Instead of exposing your server directly, use Cloudflare Tunnel:

```
# Install cloudflared
wget https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-
linux-amd64
sudo mv cloudflared-linux-amd64 /usr/local/bin/cloudflared
sudo chmod +x /usr/local/bin/cloudflared

# Create named tunnel
cloudflared tunnel create kuccps-ticketing

# Configure tunnel
cloudflared tunnel route dns kuccps-ticketing ticketing.kuccps.ac.ke

# Run tunnel
cloudflared tunnel run kuccps-ticketing
```

Benefits:

- No open ports on your server
- DDoS protection
- Free SSL certificates

- No need to manage firewall rules

Firewall Configuration

```
# UFW (Ubuntu)
sudo ufw allow 22/tcp      # SSH
sudo ufw allow 80/tcp      # HTTP
sudo ufw allow 443/tcp     # HTTPS
sudo ufw enable
```

Previous
« Database Deployment

Next
Production Checklist »

GitHub

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Production Checklist

Complete checklist for moving the KUCCPS IT Ticketing System from local testing to production.

Before Deployment

- [ ] Create production PostgreSQL database
- [ ] Rotate `JWT_SECRET` to a strong random value (32+ characters)
- [ ] Rotate `PUBLIC_TICKET_API_KEY` to a strong random value (32+ characters)
- [ ] Set `NODE_ENV=production`
- [ ] Set `FRONTEND_URL` and `CORS_ORIGINS` to production HTTPS URLs
- [ ] Set `REQUIRE_PUBLIC_TICKET_API_KEY=true`
- [ ] Configure production SMTP credentials
- [ ] Configure IMAP only if inbound monitoring is required
- [ ] Set `LDAP_URL` to `ldaps://` (never `ldap://`)
- [ ] Configure file upload directory with appropriate permissions
- [ ] Set `ALLOW_PUBLIC_API_KEY_IN_BODY=false`
- [ ] Set TLS rejection flags to `true` for SMTP, IMAP, LDAP
- [ ] Update `SUPPORT_EMAIL` and `EMAIL_FROM` to production addresses

Backend Deployment

- [ ] Run preflight checks: `npm run preflight`
- [ ] Generate Prisma client: `npx prisma generate`
- [ ] Apply migrations: `npx prisma migrate deploy`
- [ ] Configure process manager (PM2, systemd, or Docker)
- [ ] Configure log rotation
- [ ] Set up health check monitoring (`/health`)
- [ ] Configure reverse proxy (Nginx or Cloudflare Tunnel)
- [ ] Enable HTTPS with valid SSL certificate
- [ ] Configure firewall to allow only necessary ports
- [ ] Test health endpoint from external network

Frontend Deployment

- [ ] Set `VITE_API_URL` to production backend URL
- [ ] Build frontend: `npm run build`
- [ ] Deploy to Vercel, Netlify, or static host
- [ ] Configure custom domain with HTTPS
- [ ] Test all routes and navigation
- [ ] Verify API calls work correctly
- [ ] Test on mobile devices

Database

- [ ] Schedule automated daily backups
- [ ] Test restore procedure on a copy of production data
- [ ] Configure connection pooling if needed
- [ ] Set up database monitoring
- [ ] Document backup and restore procedures

Email

- [ ] Test ticket creation email
- [ ] Test ticket update email
- [ ] Test ticket resolution email
- [ ] Verify sender domain has SPF/DKIM/DMARC records
- [ ] Monitor bounce and spam rates
- [ ] Test IMAP monitoring if enabled

Google Workspace Add-on

- [ ] Update `API_URL` to production backend
- [ ] Update `BACKEND_API_KEY` to production key
- [ ] Create new deployment version in Apps Script
- [ ] Domain-install the add-on
- [ ] Test ticket creation from Gmail
- [ ] Verify ticket numbers are created correctly

Active Directory

- [ ] Confirm LDAP URL, bind DN, bind password
- [ ] Confirm search base and filter
- [ ] Test user sync: `POST /api/users/sync/ad`
- [ ] Test directorate sync: `POST /api/departments/sync/ad`

Test LDAP login

- [ ] Keep break-glass admin path available

Security

- [ ] Verify CORS only allows production origins
- [ ] Verify rate limiting is active
- [ ] Verify security headers are present
- [ ] Run security audit on dependencies: `npm audit`
- [ ] Review `.gitignore` to ensure `.env` is not committed
- [ ] Verify file upload restrictions
- [ ] Test authentication failure handling

Monitoring

- [ ] Set up uptime monitoring for `/health`
- [ ] Configure error logging (e.g., Sentry, Logflare)
- [ ] Set up database query monitoring

- [ ] Configure alerting for high error rates
- [ ] Monitor disk space for uploads

Documentation

- [ ] Update this checklist with final values
- [ ] Document production URLs and credentials (securely)
- [ ] Share emergency contact information
- [ ] Document rollback procedure

Go-Live

- [ ] Announce maintenance window if applicable
- [ ] Deploy backend
- [ ] Deploy frontend
- [ ] Run database migrations
- [ ] Verify health check passes
- [ ] Test core user flows end-to-end
- [ ] Monitor logs for errors
- [ ] Announce system is live

Post-Deployment

- [ ] Create initial departments

- [ ] Add staff and admin users
- [ ] Configure email templates
- [ ] Configure SLA policies
- [ ] Set up notification preferences
- [ ] Train users on the system
- [ ] Monitor for issues for 48 hours

Previous
« Reverse Proxy

Next
Maintenance »

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Maintenance

This guide covers routine maintenance tasks for the KUCCPS IT Ticketing System.

Daily Tasks

- [ ] Review open and overdue tickets
- [ ] Check failed email sends in logs
- [ ] Review critical tickets approaching SLA breach
- [ ] Verify backup completion
- [ ] Monitor system health endpoint

Weekly Tasks

- [ ] Review database backup logs
- [ ] Check knowledge base article usefulness (ratings/views)
- [ ] Review user and department changes
- [ ] Review status issue history
- [ ] Check disk space for uploads directory
- [ ] Review error logs for patterns

Monthly Tasks

- [ ] Rotate secrets where appropriate (`JWT_SECRET`, API keys)
- [ ] Update npm dependencies after testing
- [ ] Review audit logs and ticket history
- [ ] Test restore procedures
- [ ] Review SLA policy performance
- [ ] Clean up old attachments if storage is constrained
- [ ] Review and update documentation

Dependency Updates

```
# Backend
cd backend
npm outdated
npm update

# Frontend
cd frontend
npm outdated
npm update

# Check for vulnerabilities
cd backend && npm audit
cd frontend && npm audit
```

Database Maintenance

Vacuum Analyze

```
sudo -u postgres psql -d kuccps_ticketing -c "VACUUM ANALYZE;"
```

Reindex

```
sudo -u postgres psql -d kuccps_ticketing -c "REINDEX DATABASE kuccps_ticketing;"
```

Check Table Sizes

```
SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname, tablename)) AS total_size
FROM pg_tables
WHERE schemaname = 'public'
ORDER BY pg_total_relation_size(schemaname, tablename) DESC;
```

Log Rotation

Configure log rotation for application logs:

```
# /etc/logrotate.d/kuccps-ticketing
/var/log/kuccps-ticketing/*.log {
    daily
    rotate 30
    compress
```

```
    delaycompress
    notifempty
    create 644 www-data www-data
    postrotate
        systemctl reload kuccps-backend
    endscript
}
```

Performance Tuning

Database Indexes

Monitor slow queries and add indexes as needed:

```
EXPLAIN ANALYZE SELECT * FROM "Ticket" WHERE "status" = 'Open' AND "createdAt" > NOW()
- INTERVAL '30 days';
```

Prisma Query Optimization

- Use `select` to fetch only needed columns
- Use `include` selectively for relations
- Avoid N+1 queries by using `include` or separate `findMany` calls

« Production Checklist

Next
Backup and Restore »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Backup and Restore

This guide covers backup strategies and restore procedures for the KUCCPS IT Ticketing System.

Database Backups

Automated Daily Backups

Set up a cron job for automated backups:

```
# Edit crontab
crontab -e

# Add daily backup at 2:00 AM
0 2 * * * /usr/local/bin/backup-db.sh
```

Create `/usr/local/bin/backup-db.sh`:

```
#!/bin/bash
BACKUP_DIR=/backups/kuccps
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
mkdir -p $BACKUP_DIR
```

```
# Database backup
pg_dump -U kuccps_user kuccps_ticketing | gzip > $BACKUP_DIR/db_$TIMESTAMP.sql.gz

# Keep only last 30 days
find $BACKUP_DIR -name "db_*.sql.gz" -mtime +30 -delete

# Upload to S3 (optional)
aws s3 cp $BACKUP_DIR/db_$TIMESTAMP.sql.gz s3://kuccps-backups/
```

Manual Backup

```
# Full backup
pg_dump -U kuccps_user kuccps_ticketing > db_backup.sql

# Compressed backup
pg_dump -U kuccps_user kuccps_ticketing | gzip > db_backup.sql.gz

# Specific tables
pg_dump -U kuccps_user -d kuccps_ticketing -t Ticket -t User > tables_backup.sql
```

File Backups

Uploads are stored in the filesystem (default: `./uploads`). Back them up separately:

```
# Tar the uploads directory
tar -czf uploads_backup.tar.gz ./uploads

# Upload to S3
aws s3 cp uploads_backup.tar.gz s3://kuccps-backups/
```

Restore Procedure

Restore Database

```
# Stop the backend
pm2 stop kuccps-backend

# Restore from backup
gunzip -c db_backup.sql.gz | psql -U kuccps_user -d kuccps_ticketing

# Start the backend
pm2 start kuccps-backend
```

Restore Files

```
# Stop the backend
pm2 stop kuccps-backend

# Restore uploads
tar -xzf uploads_backup.tar.gz

# Set permissions
chown -R www-data:www-data ./uploads

# Start the backend
pm2 start kuccps-backend
```

Disaster Recovery Plan

1. **RTO (Recovery Time Objective):** 2 hours
2. **RPO (Recovery Point Objective):** 24 hours (daily backups)
3. **Testing:** Monthly restore tests on staging environment

Recovery Steps

1. Provision new server or restore existing
2. Install Node.js, PostgreSQL, and dependencies
3. Restore database from latest backup
4. Restore file uploads
5. Configure environment variables
6. Deploy backend and frontend
7. Run migrations if needed
8. Verify health check
9. Test core functionality
10. Update DNS if server IP changed

Offsite Backups

Store backups in a different physical location:

- AWS S3 with versioning
- Google Cloud Storage
- Offsite SFTP server
- Encrypted email attachments (for small databases)

Encrypt sensitive backups:

```
# Encrypt with GPG
gpg --symmetric --cipher-algo AES256 db_backup.sql.gz

# Decrypt
gpg --decrypt db_backup.sql.gz.gpg > db_backup.sql.gz
```

Previous
« Maintenance

Next
Monitoring »

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Monitoring

This guide covers monitoring the health and performance of the KUCCPS IT Ticketing System.

Health Check

Endpoint

```
GET /health
```

Response

```
{
  "status": "OK",
  "message": "KUCCPS Ticketing System API is running"
}
```

Monitoring Tools

Use external monitoring services to check the health endpoint:

Tool	Setup
UptimeRobot	Free HTTP monitor
Pingdom	Advanced monitoring with alerts
Datadog	Full-stack observability
Grafana + Prometheus	Self-hosted metrics

Application Metrics

Key Metrics to Track

Metric	Description	Alert Threshold
API response time	Average request latency	> 500ms
Error rate	5xx responses per minute	> 5
Ticket creation rate	Tickets created per hour	Anomaly detection
Active users	Concurrent authenticated users	N/A
Database connections	Active DB connections	> 80% of pool
Disk usage	Upload directory size	> 80%

Logging

Log important events with request IDs:

```
console.error('Unhandled request error:', { requestId: req.id, message: err.message });
```

Log format should include:

- Timestamp
- Request ID
- Log level (info, warn, error)
- Message
- Context (user ID, ticket ID, etc.)

Database Monitoring

Connection Count

```
SELECT count(*) FROM pg_stat_activity WHERE datname = 'kuccps_ticketing';
```

Slow Queries

Enable slow query logging in `postgresql.conf`:

```
log_min_duration_statement = 1000  # Log queries > 1s
```

Table Size

```
SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname, tablename)) AS total_size
FROM pg_tables
WHERE schemaname = 'public'
ORDER BY pg_total_relation_size(schemaname, tablename) DESC;
```

Email Monitoring

Monitor email delivery:

- Check SMTP logs for bounces and failures
- Monitor IMAP connection health
- Track email queue depth
- Alert on consecutive failed sends

Alerting

Set up alerts for:

1. **Critical:** Health check failure, database down, error rate spike
2. **Warning:** High response time, approaching disk capacity, failed email sends
3. **Info:** Daily digest sent, backup completed

Alert Channels

- Email to administrators
- Slack/Discord webhook
- SMS for critical alerts
- PagerDuty for on-call escalation

Previous
« Backup and Restore

Next
Common Issues »

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Common Issues

This guide covers common issues encountered when running or deploying the KUCCPS IT Ticketing System.

Issue Categories

- CORS Issues
- Database Issues
- Email Issues
- Authentication Issues
- Deployment Issues

Quick Diagnostics

Check Backend Health

```
curl http://localhost:5000/health
```

Check Frontend Build

```
cd frontend
npm run build
npm run preview
```

Check Database Connection

```
cd backend
npx prisma studio
```

Check Logs

```
# Backend logs
cd backend
node src/server.js

# PM2 logs
pm2 logs kuccps-backend

# Docker logs
docker logs kuccps-backend
```

Check Environment Variables

```
# Backend
cd backend
node -e "require('dotenv').config(); console.log(process.env)"
```

```
# Frontend
cd frontend
node -e "console.log(import.meta.env)"
```

Diagnostic Checklist

When troubleshooting, verify:

1. ☐ Backend is running on expected port
2. ☐ Frontend `VITE_API_URL` points to correct backend URL
3. ☐ `DATABASE_URL` is correct and database is accessible
4. ☐ `JWT_SECRET` is set and unchanged
5. ☐ CORS `FRONTEND_URL` includes the current frontend origin
6. ☐ SSL certificates are valid (production)
7. ☐ Firewall allows necessary ports
8. ☐ PostgreSQL is running and accepting connections
9. ☐ No `.env` file typos or missing values
10. ☐ Prisma Client is generated: `npx prisma generate`

Previous
« Monitoring

Next
CORS Issues »

Copyright © 2026 KUCCPS. Built with Docusaurus.

CORS Issues

This guide covers Cross-Origin Resource Sharing (CORS) issues and their resolution.

Symptoms

- Browser console shows: `No 'Access-Control-Allow-Origin' header is present`
- Frontend requests fail with `TypeError: Failed to fetch`
- Network tab shows CORS error on preflight requests

Causes

Missing Origin in CORS Config

The backend only allows origins listed in `CORS_ORIGINS` or `FRONTEND_URL`.

```
# Correct
FRONTEND_URL=http://localhost:5173,http://localhost:5174

# Wrong - missing port or wrong origin
FRONTEND_URL=http://localhost:5173
```

Wildcard Origin in Production

`validateProductionConfig()` rejects wildcard CORS origins when `NODE_ENV=production`.

```
# Wrong in production
FRONTEND_URL=*

# Correct in production
FRONTEND_URL=https://ticketing.kuccps.ac.ke
```

HTTP vs HTTPS Mismatch

If the frontend is served over HTTPS and the backend over HTTP, the browser considers them different origins.

```
# Frontend
VITE_API_URL=https://api.kuccps.ac.ke/api

# Backend
FRONTEND_URL=https://ticketing.kuccps.ac.ke
```

Resolution Steps

1. **Check the browser console** for the exact origin being blocked.
2. **Verify** `FRONTEND_URL` in `backend/.env` includes the frontend origin.
3. **Restart the backend** after changing `.env`.
4. **Check for typos** in the origin URL (protocol, host, port).
5. **Verify reverse proxy** is forwarding the correct `Host` header.

Testing CORS

```
# Test preflight request
curl -X OPTIONS http://localhost:5000/api/tickets \
  -H "Origin: http://localhost:5173" \
  -H "Access-Control-Request-Method: GET" \
  -v
```

Expected response headers:

```
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization, X-Requested-With
```

Nginx CORS Headers

If using Nginx as a reverse proxy, ensure it passes the `Origin` header:

```
location /api {
    proxy_pass http://localhost:5000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Previous
« Common Issues

Next
Database Issues »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Database Issues

This guide covers common database-related issues.

Symptoms

- `PrismaClientInitializationError`
- `Connection refused` errors
- `Database is out of sync`
- Slow query performance
- Migration failures

Common Issues

Prisma Client Not Generated

Symptom: `PrismaClientInitializationError: Prisma Client could not be located`

Resolution:

```
cd backend
```

```
npx prisma generate
```

Database Connection Refused

Symptom: `ECONNREFUSED` or `Connection refused`

Resolution:

1. Verify PostgreSQL is running:

   ```
   sudo systemctl status postgresql
   ```

2. Check `DATABASE_URL` in `.env`:

   ```
   DATABASE_URL="postgresql://user:password@localhost:5432/database_name?
   schema=public"
   ```

3. Verify credentials and database exist:

   ```
   psql -U user -d database_name
   ```

Schema Out of Sync

Symptom: `Migration failed to apply` or missing columns/tables

Resolution:

```
cd backend
```

```
npx prisma migrate dev
```

For production:

```
cd backend
npx prisma migrate deploy
```

Slow Queries

Diagnosis:

```
cd backend
npx prisma studio
```

Use the Query tab to test queries and check performance.

Resolution:

1. Add indexes on frequently filtered fields
2. Use `select` to limit fetched columns
3. Use `include` selectively
4. Avoid `findMany` without filters on large tables

Duplicate Ticket Numbers

Symptom: Two tickets have the same `ticketNumber`

Resolution:

The system uses a mutex to prevent this under concurrent load. If duplicates occur:

1. Check mutex implementation in `services/`
2. Ensure the server is running as a single instance (not multiple replicas without distributed locking)

Disk Space

Symptom: `No space left on device`

Resolution:

```
# Check disk usage
df -h

# Clean up old uploads
find ./uploads -type f -mtime +90 -delete

# Vacuum database
sudo -u postgres psql -d kuccps_ticketing -c "VACUUM;"
```

Previous
« CORS Issues

Next
Email Issues »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Email Issues

This guide covers email-related issues and their resolution.

Symptoms

- Emails not sent
- "Invalid login" errors from SMTP
- IMAP connection failures
- Duplicate or missing notifications

SMTP Issues

Invalid Credentials

Symptom: `Invalid login: 535-5.7.8 Username and Password not accepted`

Resolution:

1. Verify `EMAIL_USER` and `EMAIL_PASSWORD` in `.env`
2. For Gmail, use an **app password**, not your account password

3. Ensure 2FA is enabled on the Gmail account
4. Check that "Less secure app access" is enabled (if not using app passwords)

Connection Refused

Symptom: `Connection refused` or `ETIMEDOUT`

Resolution:

1. Check firewall rules allow outbound SMTP (port 587/465)
2. Verify `SMTP_HOST` and `SMTP_PORT` are correct
3. Test connectivity:

```
telnet smtp.gmail.com 587
```

TLS/SSL Errors

Symptom: `TLS handshake failed`

Resolution:

1. Set `SMTP_TLS_REJECT_UNAUTHORIZED=false` for testing only
2. In production, use valid certificates
3. For Gmail, use port 587 with `secure: false` or port 465 with `secure: true`

IMAP Issues

Connection Failures

Symptom: IMAP monitoring does not start

Resolution:

1. Verify `IMAP_HOST`, `IMAP_PORT`, `IMAP_USER`, `IMAP_PASSWORD`
2. Ensure both `EMAIL_USER` and `IMAP_USER` are configured
3. Check firewall allows outbound IMAP (port 993)

Email Not Processed

Symptom: Inbound emails do not create ticket updates

Resolution:

1. Check email subject contains ticket number
2. Verify `IMAP_POLL_INTERVAL` is appropriate
3. Check backend logs for IMAP errors
4. Ensure the mailbox is not full

Email Not Delivered

Spam/Junk Folder

Resolution:

1. Configure SPF, DKIM, and DMARC records for `SUPPORT_EMAIL` domain
2. Use a dedicated support mailbox, not a personal Gmail
3. Avoid spam trigger words in email templates

Gmail Specific

Resolution:

1. Enable 2FA
2. Generate an app password
3. Use the app password in `EMAIL_PASSWORD`

Testing Email

```
# Backend test endpoint
curl -X POST http://localhost:5000/api/notifications/test \
  -H "Authorization: Bearer <jwt-token>"
```

Monitoring

Monitor email delivery:

- Check SMTP server logs
- Monitor bounce rates
- Track email queue depth
- Set up alerts for consecutive failures

Previous
« Database Issues

Next
Authentication Issues »

Copyright © 2026 KUCCPS. Built with Docusaurus.

Authentication Issues

This guide covers authentication and authorization issues.

Symptoms

- Login fails with "Invalid credentials"
- "Token expired" errors
- 403 Forbidden on protected routes
- Users cannot log in after AD sync

Login Failures

Invalid Credentials

Symptom: `Invalid credentials`

Resolution:

1. Verify username and password are correct

2. Check `bcrypt` hash comparison in auth middleware
3. Verify `JWT_SECRET` is set and unchanged
4. Check database for the user record:

```
cd backend
npx prisma studio
```

LDAP Authentication Failures

Symptom: LDAP users cannot log in

Resolution:

1. Verify `USE_LDAP_AUTH=true`
2. Check `LDAP_URL` uses `ldaps://` in production
3. Verify `LDAP_BIND_DN` and `LDAP_BIND_PASSWORD`
4. Check `LDAP_SEARCH_BASE` and `LDAP_SEARCH_FILTER`
5. Test LDAP connection:

```
ldapsearch -x -H ldaps://ldap.example.ac.ke:636 -D
"cn=admin,dc=example,dc=ac,dc=ke" -W
```

Token Issues

Token Expired

Symptom: `Token expired` or 401 on valid requests

Resolution:

1. Check `JWT_EXPIRES_IN` setting
2. Frontend should handle token refresh or redirect to login
3. Client should clear token on 401:

```
if (response.status === 401) {
  localStorage.removeItem('token');
  window.location.href = '/login';
}
```

Invalid Token

Symptom: 401 on requests with token

Resolution:

1. Verify `JWT_SECRET` has not changed
2. Check token format in `Authorization` header: `Bearer <token>`
3. Ensure token was not truncated

Authorization Issues

403 Forbidden

Symptom: Access denied on routes that should be accessible

Resolution:

1. Verify user role in database
2. Check middleware application order
3. Ensure `requireStaffOrAdmin` or `requireAdmin` is applied correctly
4. Check that the user's role matches the required role

Role Not Applied

Symptom: New users do not have expected permissions

Resolution:

1. Verify `role` field in `User` model
2. Check that role defaults are correct in `schema.prisma`
3. Verify frontend checks `currentUser.role` before rendering admin tabs

Password Issues

Password Not Hashing

Symptom: Plain-text passwords stored in database

Resolution:

1. Verify `bcrypt` is called before `prisma.user.create` or `update`
2. Check controller code for password hashing

Password Comparison Failing

Resolution:

1. Ensure both hashing and comparison use the same `bcrypt` rounds
2. Verify password is not double-hashed

Previous
« Email Issues

Next
Deployment Issues »

GitHub

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.

Deployment Issues

This guide covers common deployment-related issues.

Symptoms

- 502 Bad Gateway
- 503 Service Unavailable
- Health check fails
- Frontend shows blank page
- API calls return CORS errors in production

502/503 Errors

Symptom: Reverse proxy returns 502 or 503

Resolution:

1. Verify backend is running:

```
curl http://localhost:5000/health
```

2. Check backend process:

```
# PM2
pm2 status
pm2 logs kuccps-backend

# Docker
docker ps
docker logs kuccps-backend
```

3. Verify Nginx proxy configuration:

```
nginx -t
sudo systemctl reload nginx
```

Health Check Failures

Symptom: `/health` returns non-200 status

Resolution:

1. Check database connectivity
2. Verify `DATABASE_URL` is correct
3. Check for Prisma connection pool exhaustion
4. Review application logs for startup errors

Blank Frontend Page

Symptom: Frontend loads but shows blank page

Resolution:

1. Check browser console for JavaScript errors
2. Verify `VITE_API_URL` is correct
3. Check that `index.html` is served correctly
4. Verify SPA routing configuration in Nginx:

```
location / {
    try_files $uri $uri/ /index.html;
}
```

SSL Certificate Issues

Symptom: `ERR_SSL_PROTOCOL_ERROR` or certificate warnings

Resolution:

1. Verify certificate files exist:

```
ls -la /etc/letsencrypt/live/ticketing.kuccps.ac.ke/
```

2. Check certificate expiry:

```
openssl x509 -in /etc/letsencrypt/live/ticketing.kuccps.ac.ke/cert.pem -noout -
dates
```

3. Renew certificate:

```
sudo certbot renew
sudo systemctl reload nginx
```

Process Management Issues

PM2 Crashes

Resolution:

```
pm2 restart kuccps-backend
pm2 logs kuccps-backend --lines 100
```

Docker Container Exits

Resolution:

```
docker logs kuccps-backend
docker ps -a
```

Check for:

- Missing environment variables
- Port conflicts
- Database connection failures

Cloudflare Tunnel Issues

Symptom: Tunnel connection drops

Resolution:

1. Verify tunnel is running:

```
cloudflared tunnel info kuccps-ticketing
```

2. Check tunnel routes:

```
cloudflared tunnel route dns list
```

3. Restart tunnel:

```
cloudflared tunnel run kuccps-ticketing
```

Environment Variable Issues

Symptom: Application starts but features don't work

Resolution:

1. Compare deployed `.env` with local `.env`
2. Verify no typos in variable names
3. Ensure secrets are properly escaped (no unescaped quotes)
4. Check for duplicate keys in `.env`

Previous
« Authentication Issues

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.